Smart Document Evaluator

Smart Document Evaluator (branded in the UI as **Smart Docs Validator**) is a private academic web application built for the **IT332 / CS342** course. Students sign in with Google, upload submissions to **Supabase Storage**, and view a structured **AI evaluation report** (rubric scores, executive summary, per-page Before → After fixes, visual & diagram review). Teachers use the **grading / review queue**, class roster tools, analytics, and the optional **Google Gemini** evaluator. Invited students also receive a branded invitation email via **Resend** when they're added to the class list.

> ***Production:*** `https://www.smartformevaluator.com` *(custom domain on Vercel)* ***Vercel default URL:*** `https://smart-document-evalutator.vercel.app`

Table of contents

Complete tech stack (with version numbers)

> *Versions below are the **exact resolved versions** from* `package-lock.json` *(the lockfile checked into the repo). Run* `npm install` *to reproduce.*

Runtime / build tools

Tool	Version	Purpose
Node.js	**18 LTS** (tested up to 22.x)	JavaScript runtime
npm	**10+**	Package manager (ships with Node 18+)
TypeScript	**5.6.3**	Static typing across the codebase
Vite	**5.4.21**	Dev server + production bundler
PostCSS	**8.5.14**	CSS pipeline
Autoprefixer	**10.4.20**	CSS vendor prefixes
ESLint	**9.12.0**	Linting (flat config)
typescript-eslint	**8.8.1**	TS-aware lint rules

@eslint/js	**9.12.0**	ESLint recommended JS rules
eslint-plugin-react-hooks	**5.1.0-rc**	React hook rules
eslint-plugin-react-refresh	**0.4.12**	Vite HMR safety
globals	**15.11.0**	Globals presets for ESLint

Frontend dependencies

Package	Version	Purpose
React	**18.3.1**	UI framework
react-dom	**18.3.1**	DOM renderer
@types/react	**18.3.11**	React types
@types/react-dom	**18.3.0**	React-DOM types
react-router-dom	**7.15.0**	Client-side routing
@vitejs/plugin-react	**4.3.2**	Vite React plugin
Tailwind CSS	**3.4.17**	Utility CSS
lucide-react	**0.344.0**	Icon set
mammoth	**1.12.0**	Browser-side `.docx` parsing
@supabase/supabase-js	**2.57.4**	Supabase browser/server SDK

Backend / infrastructure (managed services + serverless)

Service / library	Version	Role
Supabase Postgres	**15.x** (managed)	App database (Row-Level-Security)
Supabase Auth	Current (Gotrue v2)	Google OAuth (PKCE)
Supabase Storage	Current	Student upload bucket
Vercel Functions	Node runtime (latest)	Serverless `api/send-invitation-email.ts`
Resend	API v1	Transactional email (DKIM-signed)
nodemailer	**8.0.7**	Gmail SMTP fallback (`invite:gmail`)
Google Gemini	`gemini-2.5-flash` (configurable)	Optional AI evaluator
Google Forms	n/a	Optional Rate-us survey backend

CLI / dev tooling (devDependencies)

Package	Version	Purpose
pg	**8.20.0**	Postgres client for `npm run db:apply`

sharp	**0.34.5**	Mascot PNG matte removal (`npm run mascot:strip-bg`)

Hosting / DNS

Component	Provider	Notes
Frontend hosting	Vercel (free / Hobby)	Auto-deploys from `main`
Domain registrar	GoDaddy	Domain: `smartformevaluator.com`
DNS	Vercel nameservers (`ns1.vercel-dns.com`, `ns2.vercel-dns.com`)	Auto-issues SSL via Let's Encrypt
Email DNS records	Vercel DNS	Resend domain `send.smartformevaluator.com` (DKIM + SPF MX + SPF TXT)

Architecture (high level)

```
flowchart LR
  subgraph client [Browser SPA]
    UI[React pages + Layout]
    Auth[AuthContext + access gate]
    Store[Supabase client + uploads]
    AI[Gemini eval + AIDocumentEvaluationReport]
    Notifier[InvitedStudentEmailNotifier]
  end
  subgraph vercel [Vercel]
    SPA[Static SPA]
    Fn["/api/send-invitation-email"]
  end
  subgraph supa [Supabase]
    PG[(Postgres + RLS)]
    ST[Storage bucket]
    SA[Auth / Google OAuth]
  end
  subgraph google [Google services]
    GEM[Gemini API or custom eval URL]
    FORM[Google Forms Rate us]
  end
  subgraph mail [Email]
    RS[Resend transactional]
    SMTP[Gmail SMTP fallback]
  end
  UI --> Auth
  UI --> Store
  UI --> AI
  UI --> Notifier
  Notifier --> Fn
  Fn --> RS
  Auth --> SA
```

```
Store --> PG
Store --> ST
AI --> GEM
UI --> FORM
SPA -.-> UI
SMTP -.fallback.-> RS
```

- **Single-page app (SPA)** — all routes render inside `Layout` after sign-in; role (`student` | `teacher` | `admin`) gates which routes exist.
- **Access gate** — student sign-in is restricted to the gmails in `src/data/invitedStudentEmails.ts` . Teachers / admins are whitelisted via env vars.
- `VITE_*` values are inlined at **build time**; **server-only** values (`RESEND_API_KEY` , `SMARTDOCS_FROM_EMAIL` , …) are read at request time by the Vercel serverless function only.

Features

Area	Description
Authentication	Supabase Auth with **Google OAuth** (PKCE).
Roles	`student`, `teacher`, `admin`. `VITE_ADMIN_EMAILS` / `VITE_TEACHER_EMAILS` elevate by email; everything else is a student.
Class-list access gate	Students whose Gmail isn't on the official roster are signed out immediately with a friendly message — the app shell is never mounted for unauthorized accounts.
Students	Dashboard, Submit work, My Submissions, Tasks, Boards, Calendar, Drive, Sheets, Analytics, **TEAM 14**, Settings.
Student UX	Floating **Rate us** button + **Eva** anime onboarding tour.
Teachers	Dashboard, **Grading** (review queue), Student Submissions, Documents, Analytics, Class list, Instructions/Inbox, Settings.
AI grading (optional)	**Run AI Evaluator** — heuristic fallback when Gemini is unavailable; otherwise structured rubric + executive summary + per-page Before/After + diagram review + multimodal (PDF / image / audio / video) attachments.
Invitation emails	Branded HTML email sent **once per user** when an invited student signs in for the first time; also bulk-sendable via CLI.
Bulk actions	Multi-select + "Delete selected" controls on class list, student submissions, grading, and submission-status pages.
Stable UI	No flicker on alt-tab or route swap; background fetches are throttled.

Test accounts / sample credentials

Sign-in is **Google OAuth only** — there is no traditional email-and-password screen. The "password" is always the Google account's own password (we never see or store it). To test the system as a particular role, you must sign into Google with an account whose email matches one of the lists below.

> ⚠️ *Do not commit real passwords to git. If you need to share working credentials with an evaluator, send them out-of-band (email, encrypted message) and rotate after the evaluation window.*

Live (production) test accounts on `https://www.smartformevaluator.com`

Role	Sign-in email (Google)	Password	Notes

Admin	*(fill in your admin Gmail — must be listed in `VITE_ADMIN_EMAILS`)*	*Google account password*	Full access to every page and to teacher tools.
Teacher	*(fill in your teacher Gmail — must be listed in `VITE_TEACHER_EMAILS`)*	*Google account password*	Sees teacher Dashboard, Grading, Class list, Analytics.
Student (project owner)	`trafalgardreii@gmail.com`	*Google account password*	On the official class list; can submit work, see AI feedback, click Rate us.
Student (cohort sample)	`anaclaireellen@gmail.com` *(or any address from `src/data/invitedStudentEmails.ts`)*	*Google account password*	Only the owner of that Gmail can log in — share their own credentials separately.
Blocked sample	*any other Gmail*	*Google account password*	Should be auto-signed-out and shown: *"Smart Docs is for IT332 / CS342 students on the official class list only…"*

Local-dev sample roles

When running `npm run dev`, the same Google OAuth flow applies. Configure the role-mapping env vars in your `.env`:

```
VITE_ADMIN_EMAILS=alice.admin@gmail.com
VITE_TEACHER_EMAILS=bob.teacher@gmail.com,carol.teacher@gmail.com
```

Any Gmail not in those lists, but listed in `src/data/invitedStudentEmails.ts`, signs in as a **student**.

Convenience: how to add a temporary evaluator account

1. Edit `src/data/invitedStudentEmails.ts` — add the evaluator's Gmail (alphabetical).
2. Mirror the same Gmail in the server-side allow-list at the top of `api/send-invitation-email.ts`.
3. Commit + push → Vercel auto-deploys.
4. The evaluator can now sign in at `https://www.smartformevaluator.com` with their Google account.

To grant **teacher** access instead, add the Gmail to **Vercel → Settings → Environment Variables →** `VITE_TEACHER_EMAILS` (comma-separated), then redeploy.

Database export / dump

The latest schema lives in `docs/` (per-feature SQL files). To export a **complete dump** of the live Supabase database (schema **and** data), use the bundled helper:

```
# 1. Set DATABASE_URL (Supabase → Project Settings → Database → Connection string)
$env:DATABASE_URL = "postgresql://postgres.<ref>:<PASSWORD>@db.<ref>.supabase.co:5432/postgres"

# 2. Generate the dump (requires pg_dump on PATH)
npm run db:dump                      # full: schema + data
npm run db:dump -- --schema-only     # schema only
npm run db:dump -- --data-only       # data only
npm run db:dump -- --out=path\to\custom.sql
```

Output is written to `db-dumps/smart-docs-<UTC-timestamp>.sql` (folder created if missing). See `db-dumps/README.md` for restore instructions.

> 📄 ***Why isn't the dump committed to git?*** *It contains real student PII (emails, submissions, scores). The `db-dumps/` folder is gitignored. Share the file out-of-band (USB, encrypted Drive folder, email attachment) with whoever needs the snapshot. Submit a freshly generated dump with each deliverable.*

Installing `pg_dump`

OS	Command
Windows	Install **PostgreSQL Command Line Tools** from https://www.postgresql.org/download/windows/ (untick "PostgreSQL Server", keep "Command Line Tools"). Make sure `pg_dump --version` works in a new terminal.
macOS	`brew install libpq && brew link --force libpq`
Linux (Debian/Ubuntu)	`sudo apt-get install postgresql-client`

Schema-only reference

If you only need the canonical schema (no data), the files under `docs/` make a self-contained schema dump:

File	Purpose
`docs/supabase-setup-all-in-one.sql`	Consolidated bootstrap (recommended starting point)
`docs/supabase-bootstrap-public-users.sql`	`public.users` table + auth trigger
`docs/supabase-users-table.sql`	Standalone `users` table definition
`docs/supabase-assignments-submissions-core.sql`	Assignments + submissions core schema
`docs/supabase-storage-student-submissions.sql`	Bucket + policies for student uploads
`docs/supabase-rls-*.sql`	All RLS policies (per-feature)
`docs/supabase-fix-users-rls-recursion.sql`	Hotfix for recursive RLS on `users`

Deployment instructions

The system is deployed in two halves:

- **Frontend** = a static SPA bundle on **Vercel** (auto-deploys from GitHub).
- **Backend** = managed services (Supabase + Resend + optional Gemini) + one Vercel serverless function for transactional email.

Frontend (Vercel)

1. **Push the repo to GitHub**

   ```
   git remote add origin https://github.com/<you>/<repo>.git
   git push -u origin main
   ```

2. **Import the repo on Vercel** at https://vercel.com/new. Pick the GitHub repo, accept the auto-detected **Vite** preset (Build command `npm run build`, Output `dist`).

3. **Set environment variables** in **Vercel** → **Project** → **Settings** → **Environment Variables**. Apply each to all three environments (**Production**, **Preview**, **Development**):

Key	Example value
`VITE_SUPABASE_URL`	`https://<ref>.supabase.co`
`VITE_SUPABASE_ANON_KEY`	`eyJhbGciOi…`
`VITE_ADMIN_EMAILS`	`admin@example.com`
`VITE_TEACHER_EMAILS`	`teacher1@example.com,teacher2@example.com`
`VITE_SUBMISSION_STORAGE_BUCKET`	`student-submissions`
`VITE_GEMINI_API_KEY` *(optional, dev only)*	`AIza…`
`VITE_GEMINI_MODEL` *(optional)*	`gemini-2.5-flash`
`RESEND_API_KEY`	`re_…`
`SMARTDOCS_FROM_EMAIL`	`Smart Docs <noreply@send.smartformevaluator.com>`
`SMARTDOCS_APP_URL`	`https://www.smartformevaluator.com`
`SMARTDOCS_SURVEY_URL` *(optional)*	Google Form URL

4. **Custom domain** (optional, but used in production): **Vercel** → **Domains** → **Add** `smartformevaluator.com` and `www.smartformevaluator.com`. Vercel will print the two nameservers to set at your registrar:

```
ns1.vercel-dns.com
ns2.vercel-dns.com
```

   At **GoDaddy** (or your registrar) → **DNS** → **Nameservers** → **Change Nameservers** → **"Enter my own nameservers"** → paste the two above → **Save**. Vercel will auto-detect within 5–30 minutes, mark the domain green, and issue SSL via Let's Encrypt.

5. **SPA + serverless routing** is already configured in `vercel.json`:

```
{ "rewrites": [{ "source": "/((?!api/).*)", "destination": "/index.html" }] }
```

   This rewrites all non-API routes to `index.html` (so React Router can take over) while letting `/api/*` resolve to serverless functions.

6. **Trigger the first deploy** (push any commit, or click **Redeploy** in Vercel). Confirm the build log ends with "Production: Ready" and the assigned `*.vercel.app` URL works.

7. **Smoke test**:

```
curl -I https://<project>.vercel.app                      # → 200
curl -I https://www.smartformevaluator.com                # → 200 (after DNS propagates)
curl -sX POST https://<project>.vercel.app/api/send-invitation-email   # → 405 or 400 (function is
reachable)
```

Backend (Supabase + Resend + Gemini)

1. Supabase project

1. Create a Supabase project at https://supabase.com/dashboard.
2. **Authentication → Providers → Google** → enable, paste a Google OAuth Client ID + Secret (from https://console.cloud.google.com/apis/credentials). Authorised redirect URI for the OAuth client:

   ```
   https://<ref>.supabase.co/auth/v1/callback
   ```

3. **Authentication → URL Configuration**:
   - **Site URL:** `https://www.smartformevaluator.com`
   - **Redirect URLs** (add all):

     ```
     https://www.smartformevaluator.com/**
     https://smartformevaluator.com/**
     https://<your-vercel-project>.vercel.app/**
     http://localhost:5173/**
     ```

4. **Database → SQL Editor** → run `docs/supabase-setup-all-in-one.sql` (creates `public.users`, assignments, submissions, RLS policies). If your fresh project has a stricter `auth.users` trigger, also run `docs/supabase-bootstrap-public-users.sql` and `docs/supabase-fix-users-rls-recursion.sql`.
5. **Storage → Create bucket →** `student-submissions` (private). Then in the SQL editor run `docs/supabase-storage-student-submissions.sql`.
6. **Project Settings → API** → copy the **Project URL** into `VITE_SUPABASE_URL` and the **anon public key** into `VITE_SUPABASE_ANON_KEY` (back in Vercel env vars).

2. Resend (transactional email)

> *Walkthrough:* *`docs/INVITATION_EMAIL_SETUP.md`*

1. Sign up at https://resend.com (free tier: 100 emails/day, 3 000/month).
2. **Domains → Add Domain →** `send.smartformevaluator.com` **→ Manual setup**. Resend prints 3 DNS records (1 DKIM TXT, 1 SPF MX, 1 SPF TXT).
3. Add those 3 records in **Vercel → Domains → smartformevaluator.com → DNS Records** (using the exact `Name`, `Type`, `Value`, and `Priority=10` for the MX row).
4. Back in Resend, click **"I've already added the records"**. The badge flips to **Verified** within 5–30 min. If it sticks on Pending for over an hour, delete + re-add the domain in Resend — the DKIM key is stable per account so the existing DNS rows usually still work.
5. **API Keys → Create API Key** → copy the `re_…` value → paste into **Vercel env var** **`RESEND_API_KEY`**.
6. Set **`SMARTDOCS_FROM_EMAIL`** to a verified-domain address, e.g.:

   ```
   Smart Docs <noreply@send.smartformevaluator.com>
   ```

7. Redeploy in Vercel so the serverless function picks up the new env vars.

3. Google Gemini (optional)

Two options:

Option	When	How
Direct REST (dev only)	Local `npm run dev`	Get a key at https://aistudio.google.com/app/apikey; set `VITE_GEMINI_API_KEY` + optional `VITE_GEMINI_MODEL` in `.env`.
Built-in Vercel proxy (recommended)	Production on Vercel	Add **`GEMINI_API_KEY`** (server secret, not `VITE_`) in Vercel → Environment Variables and redeploy. The built app calls same-origin **`POST /api/gemini-evaluate`** — the key never ships in the JS bundle and Google "HTTP referrer" restrictions cannot block your custom domain or `*.vercel.app`.

Custom proxy	Any host	Set `VITE_GEMINI_EVAL_URL` to an HTTPS endpoint that accepts `POST { docType, content, template, attachments?, model? }` and returns the same JSON shape as Gemini.

Without a dev key or production `GEMINI_API_KEY`, the AI evaluator still loads — it falls back to a heuristic rubric draft generated in the browser (often ~2% on PDFs with no extracted text).

4. Google Forms (optional — Rate us survey)

- Survey content: `docs/SoftwareUsabilitySurvey-StudentRateUs.md`
- Auto-generator (Apps Script): `scripts/google-forms/create-rate-us-form.gs` — paste into https://script.google.com, run `createSmartDocsValidatorSurvey`, copy the form URL into `VITE_STUDENT_RATE_US_URL`.

5. Verify everything

```
npm run verify:env             # checks Supabase env vars
npm run verify:gemini          # optional Gemini connectivity test
npm run invite:test            # sends one Resend test email
```

Local development quick-start

```
git clone https://github.com/<you>/Smart_Document_Evaluator.git
cd Smart_Document_Evaluator
npm install

# 1. Copy and fill env (use the values from your Supabase project)
cp .env.example .env               # Windows: copy .env.example .env

# 2. Validate
npm run verify:env

# 3. Apply Supabase schema (one-time, requires DATABASE_URL in .env)
npm run db:apply
npm run db:fix-rls                 # only if profile loads fail at sign-in

# 4. Run
npm run dev
```

Open http://localhost:5173. In Supabase **Authentication** → **URL Configuration** add `http://localhost:5173/**` under **Redirect URLs** so Google sign-in works locally.

Environment variables

Client (build-time, prefixed `VITE_*`)

Variable	Required	Purpose
`VITE_SUPABASE_URL`	**Yes**	Supabase project URL
`VITE_SUPABASE_ANON_KEY`	**Yes**	Publishable anon key (RLS must protect data)
`VITE_ADMIN_EMAILS`	No	Comma-separated emails → `admin` role

`VITE_TEACHER_EMAILS`	No	Comma-separated emails → `teacher` role
`VITE_STUDENT_EMAIL_DOMAINS`	No	Restrict which domains resolve as students
`VITE_SUBMISSION_STORAGE_BUCKET`	No	Storage bucket for uploads (default `student-submissions`)
`VITE_STUDENT_RATE_US_URL`	No	Override URL for the student **Rate us** button
`VITE_GEMINI_EVAL_URL`	No	Optional custom POST proxy (same JSON contract as Gemini) — overrides the built-in `/api/gemini-evaluate` path
`VITE_GEMINI_API_KEY`	No	**Local dev only** — calls Google from the browser; visible in the built JS bundle
`VITE_GEMINI_MODEL`	No	Model id (e.g. `gemini-2.5-flash`) — sent to `/api/gemini-evaluate` and used for direct dev calls

Server-only (Vercel Function — never prefixed with `VITE_`)

Variable	Used by	Purpose
`GEMINI_API_KEY`	`api/gemini-evaluate.ts`	Google AI Studio key for teacher **Run AI Evaluator** (production builds call this route; key never ships to the browser)
`RESEND_API_KEY`	`api/send-invitation-email.ts`	Resend API key (`re_...`)
`SMARTDOCS_FROM_EMAIL`	`api/send-invitation-email.ts`	Verified-domain From address
`SMARTDOCS_APP_URL`	`api/send-invitation-email.ts`	URL the "Open Smart Docs" button in the email points to
`SMARTDOCS_SURVEY_URL`	`api/send-invitation-email.ts`	URL of the Rate us Google Form

CLI-only (your machine — never commit)

Variable	Used by	Purpose
`DATABASE_URL`	`db:apply`, `db:fix-rls`, `db:dump`	Direct Postgres connection string
`GMAIL_USER`	`invite:gmail`	Sender Gmail address
`GMAIL_APP_PASSWORD`	`invite:gmail`	16-char Google App Password (create one)
`GMAIL_FROM_NAME`	`invite:gmail`	Display name (default `Smart Docs`)
`INVITE_BATCH_GAP_MS`	`invite:send-all` / `invite:gmail`	Gap between sends in ms (default 350 / 1500)

NPM scripts

Script	Purpose
`npm run dev` / `npm start`	Vite dev server (port **5173**)
`npm run build`	Production build → `dist/`

`npm run preview`	Serve `dist/` locally
`npm run lint`	ESLint
`npm run typecheck`	TypeScript check (`tsconfig.app.json`)
`npm run verify:env`	Validate `.env` shape for Supabase
`npm run verify:gemini`	Optional Gemini connectivity test
`npm run db:apply`	Apply SQL schema files (needs `DATABASE_URL`)
`npm run db:fix-rls`	Apply users RLS recursion fix SQL
`npm run db:dump`	Export the full Supabase database to `db-dumps/*.sql` (needs `pg_dump`)
`npm run supabase:setup`	Interactive Supabase setup helper
`npm run supabase:fix-rls`	Interactive RLS fix helper
`npm run mascot:strip-bg`	Regenerate `public/mascot/eva-welcome-nobg.png` (uses **sharp**)
`npm run invite:test`	Send one test invitation email via Resend
`npm run invite:send-all`	Bulk-send invitation emails via Resend to the entire class list
`npm run invite:gmail`	Bulk-send via Gmail SMTP fallback (Nodemailer)

Project layout

```
.
├── api/
│   └── send-invitation-email.ts        ← Vercel serverless: Resend transactional email
├── db-dumps/                           ← Generated SQL dumps (gitignored)
│   └── README.md                       ← How to dump / restore
├── docs/                               ← SQL bootstrap, RLS, storage policies, SRS, setup guides
├── public/                             ← Static assets, mascot PNGs
├── scripts/
│   ├── lib/invitationEmailTemplate.mjs ← Shared HTML/text template for CLI scripts
│   ├── send-invitation-email-test.mjs  ← npm run invite:test
│   ├── send-invitation-email-bulk.mjs  ← npm run invite:send-all (Resend)
│   ├── send-invitation-email-gmail.mjs ← npm run invite:gmail (Nodemailer + Gmail SMTP)
│   ├── dump-supabase-database.mjs      ← npm run db:dump
│   ├── apply-supabase-schema.mjs       ← npm run db:apply
│   ├── check-env.mjs                   ← npm run verify:env
│   ├── test-gemini.mjs                 ← npm run verify:gemini
│   ├── remove-mascot-white-matte.mjs   ← npm run mascot:strip-bg
│   ├── gen-it332-roster-snippet.mjs    ← roster TS code generator
│   └── google-forms/create-rate-us-form.gs  ← Apps Script (paste at script.google.com)
├── src/
│   ├── App.tsx                         ← Routes + access gate; AccessGateScreen while verifying
│   ├── components/
│   │   ├── Layout.tsx                  ← shell + Rate us + Eva + InvitedStudentEmailNotifier
│   │   ├── Sidebar.tsx
│   │   ├── AIDocumentEvaluationReport.tsx
```

```
│   │   ├── UserAvatar.tsx
│   │   ├── student/
│   │   │   ├── StudentRateUsButton.tsx
│   │   │   ├── StudentOnboardingTour.tsx
│   │   │   └── InvitedStudentEmailNotifier.tsx   ← headless: triggers /api/send-invitation-email
│   │   └── teacher/
│   │       ├── TeacherWorkspaceChrome.tsx
│   │       ├── TeacherSubmissionRosterTable.tsx
│   │       └── TeacherViewScoreModal.tsx
│   ├── context/AuthContext.tsx         ← session, role, OAuth, campus + class-list gates
│   ├── data/
│   │   ├── invitedStudentEmails.ts     ← 45 gmails authorized to sign in
│   │   ├── it332Sem2ClassRoster.ts     ← personalized first/last names for the greeting
│   │   └── team14.ts
│   ├── lib/
│   │   ├── supabase.ts                 ← browser Supabase client
│   │   ├── geminiDocumentEvaluation.ts ← Gemini / proxy evaluation + parsing
│   │   ├── geminiAttachments.ts        ← multimodal inlineData parts
│   │   ├── sendInvitationEmail.ts      ← client → /api/send-invitation-email (once per user)
│   │   ├── classRosterCache.ts
│   │   ├── studentEmailPolicy.ts
│   │   └── teacherSubmissionLoad.ts
│   ├── pages/
│   │   ├── Login.tsx
│   │   ├── student/                    ← Dashboard, Assignments, Submissions, Tasks, Team14, …
│   │   └── teacher/                    ← Dashboard, ReviewQueue, StudentSubmissions, Team14, …
│   └── types/index.ts
├── vercel.json                         ← /api/* passthrough + SPA rewrite
├── DEPLOY.md                           ← Long-form deployment walkthrough
└── README.md (this file)
```

Class list & invitation emails

Smart Docs is currently a **private build for the IT332 / CS342 cohort**. The class list lives in one file:

```
src/data/invitedStudentEmails.ts   ← 45 gmail addresses (alphabetised)
```

The access gate (`src/context/AuthContext.tsx`)

After Google sign-in completes, the gate runs three checks. Any failure → immediate `signOut` and a friendly message on `/login`:

1. **`rejectStudentIfWrongCampusEmail`** — if `VITE_STUDENT_EMAIL_DOMAINS` is configured, students must sign in with a campus email.
2. **`rejectIfNotInvitedStudent`** — students must be on `INVITED_STUDENT_GMAILS`. Teachers / admins are exempt.
3. The app shell (Layout, sidebar, routes) is **only mounted after both checks pass** — unauthorized accounts see only a brief "Verifying access" screen.

Inviting a new student

1. Edit `src/data/invitedStudentEmails.ts` (and optionally add a row in `src/data/it332Sem2ClassRoster.ts` for a personalized greeting).
2. Mirror the same gmail in the server-side allow-list at the top of `api/send-invitation-email.ts`.
3. Commit + push → Vercel redeploys.

4. Send the email: `npm run invite:send-all -- --only=newstudent@gmail.com`.

Sending invitation emails

Command	What it does
`npm run invite:send-all`	Bulk-send via **Resend** to every student in `INVITED_STUDENT_GMAILS`. Supports `--only=`, `--skip=`, `--dry-run`. Requires `RESEND_API_KEY` + verified `SMARTDOCS_FROM_EMAIL`.
`npm run invite:test`	Send one test email to a single recipient.
`npm run invite:gmail`	Same bulk send, but over **Gmail SMTP** using Nodemailer (no Resend domain needed). Requires `GMAIL_USER` + `GMAIL_APP_PASSWORD`.

All three CLIs share the same HTML/text template (`scripts/lib/invitationEmailTemplate.mjs`), so the email is identical regardless of channel.

Security notes

- **Anon key + RLS** — the Supabase anon key is safe in the browser only when **Row Level Security** is enabled on every table and storage policy. Never expose the **service role** key or `DATABASE_URL` in the client.
- **Gemini** — on Vercel, set `GEMINI_API_KEY` (server-only). The app calls `/api/gemini-evaluate` so the key is never in the client bundle and Google API key referrer rules cannot break production while localhost still works with `VITE_GEMINI_API_KEY`. Optionally set `VITE_GEMINI_EVAL_URL` to your own HTTPS proxy instead.
- **Resend** — `RESEND_API_KEY` is a **server-only** variable. It lives in Vercel env vars (not prefixed with `VITE_`) and is read only inside `api/send-invitation-email.ts`.
- **Gmail App Password** — never commit, never share, never paste in the browser. Only used on the developer's machine for the SMTP fallback.
- **Student uploads** — use a dedicated bucket with policies scoped to the authenticated user (see storage SQL in `docs/`).
- **Class-list source of truth** — the gate is enforced both in the client (`AuthContext.rejectIfNotInvitedStudent`) and on the server (`api/send-invitation-email.ts` allow-list). Keep the two lists in sync.
- **Database dumps** — files in `db-dumps/` contain real student PII. The folder is gitignored; share dumps out-of-band only.

License

Private / unlicensed unless you add a `LICENSE` file. Update this section when you publish the project publicly.

Deploy so other people can use this app

“Public” means **hosted on the internet** (e.g. Vercel, Netlify). Visitors **do not** create a `.env` file; they open your URL.

Do not commit `.env` to Git. It is listed in `.gitignore`. Putting real keys in a public repo lets anyone abuse your Supabase project.

1. Put the same values in your host’s environment (not in Git)

Use the same variables as your local `.env`, but set them in the hosting dashboard:

Variable	Notes
`VITE_SUPABASE_URL`	Supabase → Project Settings → API
`VITE_SUPABASE_ANON_KEY`	Publishable / anon key (same page)
`VITE_ADMIN_EMAILS`	Optional, comma-separated
`VITE_TEACHER_EMAILS`	Optional
`VITE_SUBMISSION_STORAGE_BUCKET`	Optional; default `student-submissions`

Redeploy after changing env vars (`VITE_*` are baked in at **build** time).

2. Vercel — step-by-step

Before you start

- Code lives in a **GitHub** (or GitLab / Bitbucket) repository.
- `.env` is **not** pushed to Git (confirm `.gitignore` contains `.env`).
- Have your Supabase **Project URL** and **anon / publishable key** handy (same values as local `.env`).

A. Push your project to GitHub

1. Create a new repo on GitHub (empty or with a README).
2. On your PC, in the project folder:

```
git init
git add .
git commit -m "Initial commit"
git branch -M main
git remote add origin https://github.com/YOUR_USERNAME/YOUR_REPO.git
git push -u origin main
```

If `git add .` tries to add `.env`, stop and fix `.gitignore` before committing.

Vercel shows `404 NOT_FOUND` on `/login` or `/assignments`

That means the CDN did not find `index.html` for that path — almost always **wrong Output Directory** or the SPA rewrite not applied.

1. **Vercel → Project → Settings → General → Build & Development**
   - **Framework Preset:** Vite (or “Other” with build `npm run build`, output `dist`).
   - **Root Directory:** `.` (repo root with `package.json`).

- **Build Command:** `npm run build`
- **Output Directory:** `dist` (required for Vite — not `public`, not `.`).

2. Save, then **Deployments** → **Redeploy** the latest commit.

The repo's `vercel.json` sets `outputDirectory`, **`cleanUrls: false`** (required so rewrites to `/index.html` work — if `cleanUrls` is true, use destination `/index` instead), and SPA rewrites so every route serves `dist/index.html`.

B. Create a Vercel account and import the repo

1. Open vercel.com and sign in (GitHub login is easiest).
2. **Add New…** → **Project**.
3. **Import** your GitHub repository (authorize Vercel if asked).
4. Vercel usually detects **Vite**. Confirm:
   - **Framework Preset**: Vite (or "Other" with the settings below)
   - **Build Command**: `npm run build`
   - **Output Directory**: `dist`
   - **Install Command**: `npm install` (default)
5. **Do not click Deploy yet** — add env vars first (next section).

C. Environment variables (critical)

1. In the import screen, expand **Environment Variables**.
2. Add each row (same names and values as your local `.env`):

Name	Value
`VITE_SUPABASE_URL`	`https://xxxxx.supabase.co`
`VITE_SUPABASE_ANON_KEY`	your publishable key
`VITE_ADMIN_EMAILS`	optional, comma-separated
`VITE_TEACHER_EMAILS`	optional
`VITE_SUBMISSION_STORAGE_BUCKET`	optional, e.g. `student-submissions`

3. Leave **Environment** as Production (and add the same keys for **Preview** if you want preview deployments to work).
4. Click **Deploy**.

First deploy takes a minute or two. When it finishes, Vercel gives you a URL like `https://something.vercel.app`. Open it — the app should load.

D. After deploy — Supabase URLs

Google OAuth will fail until Supabase knows your live URL.

1. Supabase → **Authentication** → **URL Configuration**.
2. **Site URL**: `https://your-project.vercel.app` (your real Vercel URL).
3. **Redirect URLs**: add:

- `https://your-project.vercel.app/**`
- `http://localhost:5173/**` (keep local dev)

Save.

E. Google Cloud (only if you use "Sign in with Google")

1. Google Cloud Console → **APIs & Services** → **Credentials** → your OAuth 2.0 Client.
2. **Authorized JavaScript origins**: add `https://your-project.vercel.app`.
3. **Authorized redirect URIs**: keep **Supabase's** callback only:

   `https://YOUR_PROJECT_REF.supabase.co/auth/v1/callback`

   Do **not** replace this with your Vercel URL.

F. Changing env vars later

Vercel → your project → **Settings** → **Environment Variables**. Edit or add variables, then **Deployments** → … on latest → **Redeploy** (env changes require a new build for `VITE_*`).

G. SPA routing

This repo includes `vercel.json` so routes like `/assignments` load your React app instead of 404.

Short version

1. Push repo to GitHub (no `.env` in Git).
2. Vercel → New Project → Import → Build: `npm run build`, Output: `dist`.
3. Add all `VITE_*` variables → Deploy.
4. Supabase Auth URLs + Google OAuth origins as above.
5. Share your `https://….vercel.app` link.

3. Netlify (example)

1. New site from Git → build `npm run build`, publish directory `dist`.
2. Site settings → Environment variables → add the same `VITE_*` keys.
3. Add a `_redirects` file with `/* /index.html 200` (or a `netlify.toml` redirect rule) so the SPA falls back to `index.html` on deep links. This repo deploys on Vercel where `vercel.json` already handles that, so no `_redirects` is checked in.

4. Supabase (required for Google login on your live URL)

In **Supabase** → **Authentication** → **URL Configuration**:

- **Site URL**: your production site, e.g. `https://your-app.vercel.app`
- **Redirect URLs**: include
  `https://your-app.vercel.app/**`
  and keep local dev, e.g. `http://localhost:5173/**`

5. Google OAuth (if you use Google sign-in)

In **Google Cloud Console** → **OAuth client**:

- **Authorized JavaScript origins**: add `https://your-app.vercel.app`

- **Authorized redirect URIs**: keep Supabase's callback,
  `https://<your-project-ref>.supabase.co/auth/v1/callback`
  (not your Vercel URL)

After deployment, share **only the website URL** with students and teachers—not your `.env` file.

Import done — do these three things or login will not work

1. **Vercel → your project → Settings → Environment Variables**
   Add `VITE_SUPABASE_URL` and `VITE_SUPABASE_ANON_KEY` (exact same strings as your local `.env`). Enable for **Production** (and Preview if you use it).

2. **Deployments** → open the latest deployment → ⋯ → **Redeploy**
   (Required so the new variables are baked into the JS bundle.)

3. **Supabase → Authentication → URL Configuration**
   Set **Site URL** to `https://YOUR-PROJECT.vercel.app` and add **both** of these under Redirect URLs (Google OAuth returns with `?code=` on `/login`):
   `https://YOUR-PROJECT.vercel.app/**`
   `https://YOUR-PROJECT.vercel.app/login`
   Also keep `http://localhost:5173/**` for local dev.

If you open the live site and see an amber box about "Vercel needs Supabase keys," step 1–2 was skipped or Redeploy was missed.

Final presentation readiness — Smart Document Evaluator

Project: Smart Docs Validator (Smart Document Evaluator)
Course context: IT332 / CS342 · CIT–University
Date: May 14, 2026

This checklist maps **your department's pre-presentation requirements** to artifacts in this repository. Tick items locally when true; **adviser sign-off**, **peer evaluation**, and the **ACM research paper** are **external** to the repo—place signed PDFs / exports in your submission binder.

Technical and software requirements

#	Requirement	Evidence in this repo	Your action
1	**Adviser reviewed software** and gave go-signal for final presentation	*(not in repo)*	Obtain signed memo / email printout
2	**Usability testing** completed; results **analyzed** (findings + recommendations)	Survey blueprint: `docs/SoftwareUsabilitySurvey-StudentRateUs.md`	Run sessions → paste **analysis appendix** (or link) next to this checklist in binder
3	**Four documents** updated (**SRS, SDD, SPMP, STD**)	`docs/SRS-Smart-Document-Evaluator-v3.md` (+ detail in `SRS-Smart-Document-Evaluator-v2.md`) · `docs/SDD-Smart-Document-Evaluator-v3.md` · `docs/SPMP-Smart-Document-Evaluator-v3.md` · `docs/STD-Smart-Document-Evaluator-v3.md`	Export to **PDF/DOCX** if the course requires Word—Markdown is source of truth here
4	**Latest source** zipped from GitHub	`main` branch	`git archive` or GitHub **Download ZIP**
5	**ReadMe PDF** with: full **tech stack + versions**, **deployment** (frontend + backend), **sample accounts**	Source: root `README.md` (already has stack tables, Vercel + Supabase steps, test-account table)	Print/export `README.md` → **PDF**
6	**Database dump** (schema + data if applicable)	`npm run db:dump` → `db-dumps/*.sql` (folder gitignored)	Run dump on Supabase; attach **.sql** per instructor format

Quick commands (reference)

```
npm install
npm run typecheck
npm run build
npm run db:dump
```

Research requirements

#	Requirement	Evidence

1	**Full research paper** in **ACM** format	*(author off-repo or `docs/` if you add `RESEARCH-PAPER.pdf` later)*

Peer evaluation

#	Requirement	Evidence
1	**Peer evaluation** completed	*(forms handled by course)*

Document index (Markdown sources)

Document	Path
SRS v3.1	`docs/SRS-Smart-Document-Evaluator-v3.md`
SRS baseline detail	`docs/SRS-Smart-Document-Evaluator-v2.md`
SDD v3	`docs/SDD-Smart-Document-Evaluator-v3.md`
SPMP v3	`docs/SPMP-Smart-Document-Evaluator-v3.md`
STD v3	`docs/STD-Smart-Document-Evaluator-v3.md`
Usability survey spec	`docs/SoftwareUsabilitySurvey-StudentRateUs.md`
SQL / DB	`docs/supabase-setup-all-in-one.sql` (+ related `docs/supabase-*.sql`)

Note on Word templates

Department templates on the lab PC (`SRS TEMPLATE.doc` , `SDD TEMPLATE.docx` , `SPMP TEMPLATE.doc` , `STD TEMPLATE.doc`) should be used for **final formatting** if required: copy section text from the Markdown files above into the template styles (headings, table of contents, page numbers).

End of checklist

CEBU INSTITUTE OF TECHNOLOGY – UNIVERSITY

COLLEGE OF COMPUTER STUDIES

Software Requirements Specification (SRS)

for

Smart Document Evaluator (Smart Docs Validator) with AI Integration

Prepared by: Alexandrei Nash Dinapo · Jeffer Azcona · Jushua Peter Te · Ryan Bebiro

Document version: 3.1 (deliverable revision)
Date: May 14, 2026

Note on templates: The department Word templates (`SRS TEMPLATE.doc` , etc.) could not be imported as text (binary `.doc` / `.docx`). This Markdown file follows the **same conventional SRS outline** (IEEE 830–style sections) and is the **authoritative v3.1 requirements baseline** for the repository as deployed.

Change History

Version	Date	Description
1.0–2.x	2025–2026	Prior iterations; see `docs/SRS-Smart-Document-Evaluator-v2.md`.
3.0	May 13, 2026	Baseline aligned with Vite + React + Supabase + Vercel + Gemini (see v2 file header).
3.1	**May 14, 2026**	**Amendments:** Teacher **Course Tasks** (published assignments, optional handouts, due dates, delete/bulk-delete); student **Tasks** / **Submit Work** deep links (`openTask`); **deadline reminders** (in-app + optional browser notifications); **General Submission** bucket hidden from student task/workspace lists (still used for quick submit); **STD** document type on course tasks; nav label updates. Supersedes contradictory “teachers do not create assignments” wording from v3.0 where it conflicts with this revision.

Table of Contents

---

1. Introduction

1.1 Purpose

This SRS specifies functional and non-functional requirements for **Smart Document Evaluator** (UI brand: **Smart Docs Validator**), a web application for **IT332 / CS342** at CIT–University. Students submit coursework; teachers review AI-assisted evaluations and publish grades; optional **Google Gemini** provides rubric-aligned feedback.

Audience: developers, advisers, QA, deployers, and academic reviewers.

1.2 Scope

In scope

- Google OAuth (Supabase Auth) with **class-list access gate** and role mapping (`student` , `teacher` , `admin`).
- Student **Submit Work**, **Submission Status**, **Tasks**, **Boards**, **Calendar**, **Drive**, **Sheets**, **Analytics**, **Team 14**, **Settings**; onboarding tour (**Eva**); optional **Rate us** (Google Forms).
- Teacher **Dashboard**, **Grades** (review queue), **Student Submissions**, **Documents**, **Analytics**, **Class List**, **Instructions/Inbox**, **Settings**, **Team 14**, **Reports**.
- **Course Tasks:** teachers create published tasks (title, instructions, document type **SRS | SDD | SPMP | STD | Other**), optional **due date/time**, optional **handout** file (Supabase Storage); students see active tasks on **Tasks** and may open **Submit Work** with a task pre-selected.
- **AI evaluation** (Gemini or heuristic fallback), **two-lane** AI vs teacher grading, **redo** workflow, **CSV** export, **invitation email** pipeline (Resend + CLI fallbacks).
- Deployment on **Vercel** + data on **Supabase** (Postgres + Storage + RLS).

Out of scope

- Official LMS replacement; native mobile app stores; legal certification of AI output as sole evidence of integrity.

1.3 Definitions, acronyms, and abbreviations

Term	Definition
SPA	Single-page application (React + Vite + TypeScript).
RLS	Row-Level Security on Postgres.
Course task	Teacher-authored `assignments` row shown to students (vs. system “General Submission” bucket).
General Submission	Auto-provisioned assignment for quick uploads without selecting a named task; **hidden** from student Tasks/workspace lists but still used server-side.
Gemini proxy	Same-origin `POST /api/gemini-evaluate` using server `GEMINI_API_KEY`.

1.4 References

- IEEE Std 830-1998 — SRS recommended practice.
- `docs/SDD-Smart-Document-Evaluator-v3.md` — design description.
- `docs/SRS-Smart-Document-Evaluator-v2.md` — detailed FR baseline (see §3.1).
- `README.md` , `docs/supabase-setup-all-in-one.sql` , Google Gemini / Supabase / Vercel / Resend documentation.
- Republic Act No. 10173 — Data Privacy Act of 2012 (Philippines).

1.5 Overview

Section 2 describes users, constraints, and dependencies. Section 3 states **baseline** requirements (v2) and **v3.1 amendments**. Section 4 maps amendments to design/tests.

---

2. Overall description

2.1 Product perspective

Standalone HTTPS web app: browser → Vercel (SPA + `api/*`) → Supabase (Auth, Postgres, Storage) → optional Google Gemini and Resend. See architecture diagram in `README.md` and `docs/SDD-Smart-Document-Evaluator-v3.md`.

2.2 User characteristics

- **Student** — submits files, tracks status, uses Tasks/Boards; may receive deadline reminders.
- **Teacher** — manages class list, runs AI grading, publishes scores, manages **Course Tasks** (create/close/delete).
- **Admin** — env-driven elevated access; same tooling as teacher where applicable.

2.3 Constraints

- Google sign-in only; roster allow-list for students; `VITE_TEACHER_EMAILS` / `VITE_ADMIN_EMAILS` for staff.
- File size and type limits per `README.md` (e.g. ≤ 25 MB).
- Production Gemini key must not ship in client bundle (use `/api/gemini-evaluate`).

2.4 Assumptions and dependencies

- Stable internet; valid Supabase, Vercel, and (optional) Resend/Gemini credentials.
- Students use Google accounts listed in the class-list source used by the app gate.

---

3. Specific requirements

3.1 Baseline functional requirements (SRS v2)

The **numbered functional requirements FR-01 …** and interface requirements **SI-1 …** in:

`docs/SRS-Smart-Document-Evaluator-v2.md`

remain valid **except** where they explicitly state that teachers **do not** create or manage assignments. That sentence is **superseded** by §3.2 below for **Course Tasks** only. All other FR in v2 continue to apply (authentication, submissions, AI lane, teacher lane, invitation email, analytics, etc.).

3.2 Amendments and new requirements (v3.1)

ID	Requirement
AM-01	The system **shall** allow a teacher to **create a course task** with: required title; optional description; document type in `{ SRS, SDD, SPMP, STD, Other }`; optional due date/time; optional handout file.
AM-02	On successful create, the system **shall** persist the task in `assignments` (or `assignment` if singular table) with `teacher_id` = current user and `status = active` by default.
AM-03	If a handout file is provided, the system **shall** upload it to the configured Storage bucket under a teacher-scoped path and **shall** persist `handout_url` and `handout_file_name` when those columns exist (otherwise show a documented SQL migration path).
AM-04	The system **shall** allow the teacher to **close** or **re-open** a course task (`status` closed/active).
AM-05	The system **shall** allow the teacher to **delete** a single course task and to **delete selected** tasks (bulk), with confirmation, scoped to rows owned by that teacher.
AM-06	Students **shall** see all **active** course tasks assigned to the cohort in **Tasks** (grouped by due-date urgency) and in the shared student workspace data feed, **excluding** rows whose title matches the configured **General Submission** bucket title (quick-submit bucket remains available via **Submit Work** flows).

AM-07	From **Tasks**, the system **shall** provide a path to **Submit Work** with query `openTask=<assignmentId>` so the turn-in modal opens for that task.
AM-08	The system **shall** surface **in-app deadline reminders** for active tasks with due dates within a defined near window (e.g. 72 hours) or overdue, with dismiss persistence in `sessionStorage` where implemented.
AM-09	When the browser grants permission, the system **may** emit **browser notifications** for imminent deadlines subject to per-assignment daily deduplication.
AM-10	Database check constraint on `assignments.document_type` **shall** include **STD** alongside SRS, SDD, SPMP, Other (migration script provided in repo for existing databases).
AM-11	The **Submit Work** UI **shall** allow clearing a selected handout file before publish and **shall** provide a due-date picker with explicit **OK** confirmation when implemented as a custom control.

3.3 Non-functional requirements (additions)

ID	Requirement
NFR-AM-01	Course task delete operations **shall** respect RLS (`assignments_delete_own` or equivalent).
NFR-AM-02	Hiding the General Submission row from student workspace views **shall not** break quick submit or submission rows referencing that assignment id.
NFR-AM-03	Analytics completion metrics **shall** remain consistent when workspace-visible assignments are a subset of all assignment ids (exclude hidden bucket from denominators where applicable).

---

4. Requirements traceability

Amendment	SDD section	STD section
AM-01–05	§3.8 Course Tasks (Teacher)	TC-CT-01 … TC-CT-08
AM-06–09	§3.2 Student Submission; reminders component	TC-ST-01 … TC-ST-06
AM-10	§4 Data Design; SQL docs	TC-DB-01
AM-11	Teacher modal UI	TC-UI-01

---

5. References

- `docs/STD-Smart-Document-Evaluator-v3.md`
- `docs/SPMP-Smart-Document-Evaluator-v3.md`
- `docs/SDD-Smart-Document-Evaluator-v3.md`
- `docs/PRESENTATION-READINESS-AND-DELIVERABLES.md`

End of SRS v3.1

CEBU INSTITUTE OF TECHNOLOGY – UNIVERSITY

COLLEGE OF COMPUTER STUDIES

Software Requirements Specifications

for

Smart Document Evaluator (Smart Docs Validator) with AI Integration

Prepared by: Alexandrei Nash Dinapo Jeffer Azcona Jushua Peter Te Ryan Bebiro

Date: May 13, 2026 **Version:** 3.0 (revised to match the deployed system)

Change History

Name	Date	Reason for changes	Version
Alexandrei Nash Dinapo	October 27, 2025	Initial document	1.0
Project team	December 2025	Major pivot: Form Builder → Document Grader	2.0
Project team	May 13, 2026	Revised to match the deployed Smart Docs Validator system. Removed teacher-creates-assignments scope. Replaced MySQL / Express / GCP stack with the actual React + Supabase + Vercel + Resend + Gemini stack. Added class-list invitation gate, invitation-email pipeline, custom-domain deployment, and current role / module list.	**3.0**

Table of Contents

---

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) describes the functional and non-functional requirements of the **Smart Document Evaluator** system (branded in the UI as **Smart Docs Validator**) — an AI-assisted, web-based document evaluation platform built for the **IT332 / CS342** cohort at the Cebu Institute of Technology – University.

The system uses **Google's Gemini API** to generate context-aware rubric scores, executive summaries, per-page Before → After fixes, and visual / diagram review for student submissions. A mandatory **teacher-in-the-loop** review step lets instructors approve, adjust, or override the AI grade before it is released to the student.

This document is intended for:

- Development team members
- Project stakeholders and academic advisors
- Quality-assurance personnel and evaluators
- Faculty and students of the IT332 / CS342 cohort
- System administrators and deployers

The SRS serves as a contract between developers and stakeholders, and as a baseline for verification and validation.

1.2 Scope

Smart Document Evaluator is a web-based application that lets the IT332 / CS342 cohort:

- Accept document submissions (PDF, DOCX, TXT, and image files) from invited students
- Automatically evaluate submissions using **Google Gemini** against a fixed per-document-type rubric (SRS, SDD, SPMP, STD, Other)
- Provide teachers with a **review queue** to inspect AI results, adjust grades and feedback, and publish either the **AI lane** or the **Teacher lane** independently
- Release final grades and feedback to the originating student
- Export reviewed submissions to **CSV** and provide print-friendly evaluation sheets
- Send **invitation emails** (Resend transactional email, with Gmail SMTP fallback) to students newly added to the class roster

Important scope clarification: Teachers **do not create assignments or tasks** in this system. The roster of expected work and the per-document-type rubric is fixed by the course context. Teachers' job is to **review and grade** what students submit, not to author new assignment definitions.

System overview: The application is a single-page React app (SPA) backed by Supabase (Postgres + Auth + Storage) and deployed on Vercel under a custom domain. Sign-in is **Google OAuth only**. A class-list **access gate** restricts logins to a fixed allow-list of 45 student Gmail addresses plus whitelisted teacher / admin Gmail addresses. The core grading logic is **AI-first, teacher-final**.

Major functions:

- **Google OAuth sign-in** (PKCE) via Supabase Auth, restricted by class-list allow-list
- **Document submission** by students (PDF, DOCX, TXT, image)
- **AI evaluation** via Gemini (or a deployer-owned proxy URL) — rubric scoring, executive summary, Before/After page suggestions, diagram review
- **Teacher review queue** — view AI score, edit / override, publish lane-isolated grades
- **Student portal** — submit work, see AI score and Teacher score in separate read-only views, respond to "redo" requests
- **CSV export** of reviewed submissions
- **Invitation email pipeline** — branded transactional email sent once per invited student on first sign-in

Benefits:

- Removes manual transcription of rubric scores; the AI fills the rubric first and the teacher only adjusts
- Standardizes feedback quality across submissions
- Surfaces clear status (`submitted` , `under_review` , `reviewed` , `redo_requested`) for every submission
- Keeps the official numeric grade under instructor control even when AI is enabled

Target users:

- **Admin** — manages the role whitelist (via Vercel environment variables), the class-list source file, and the email allow-list
- **Teacher / faculty** — reviews AI grades, publishes final grades, requests redo, exports CSV
- **Student** — submits documents, views final grades and feedback, responds to redo requests

Out of scope (this revision):

- Teacher-authored assignments / tasks (the system does not include an "Assignment Builder")
- Native mobile apps (the SPA is responsive but not packaged for app stores)
- Replacement for the institution's official LMS gradebook
- Legal certification of AI outputs as sole evidence of academic integrity

1.3 Definitions, Acronyms and Abbreviations

Term	Definition
AI	Artificial Intelligence
API	Application Programming Interface
SPA	Single-page application (React + Vite)
OAuth 2.0	Open standard for access delegation (Google Sign-in)
PKCE	Proof Key for Code Exchange — OAuth flow used by Supabase Auth
Gemini API	Google's generative AI model used for document understanding and grading
Vertex AI / AI Studio	Google Cloud product surface from which Gemini keys are issued
Supabase	Backend-as-a-service: managed PostgreSQL, Auth, Storage, REST client
RLS	Row-Level Security — Postgres policies that restrict row access by role and ownership
JWT	JSON Web Token — the bearer token Supabase issues after sign-in
Vercel	Hosting provider used for the SPA and the serverless email function
Resend	Transactional email service used to deliver invitation emails
Nodemailer / SMTP	Gmail SMTP fallback used by the bulk-send CLI when Resend is unavailable
Class-list allow-list	The 45-entry Gmail roster in `src/data/invitedStudentEmails.ts`
Doc type	Label such as **SRS**, **SDD**, **SPMP**, **STD**, **Other** — fixes which rubric template the AI uses
Rubric	A fixed per-doc-type scoring template; the AI fills it and the teacher reviews it
Submission	A single file + metadata row uploaded by a student
AI draft / AI lane	Automated snapshot stored in `ai_draft_score` and `ai_draft_summary`
Teacher lane / Final grade	Official numeric grade stored in `score` and `feedback`
Review queue	The teacher dashboard list of AI-graded submissions pending instructor action
CSV export	Downloadable `.csv` grade report (a lighter alternative to xlsx)
Hardcopy	Print-friendly formatted evaluation sheet generated by the browser's print view
Eva	The anime mascot that runs the first-time student onboarding tour

Rate us	A Google Forms usability survey linked from a floating student-facing button

1.4 References

- Google Gemini API documentation – https://ai.google.dev/gemini-api/docs
- Google OAuth 2.0 documentation – https://developers.google.com/identity/protocols/oauth2
- Supabase documentation – https://supabase.com/docs
- Vercel documentation – https://vercel.com/docs
- Resend documentation – https://resend.com/docs
- Nodemailer documentation – https://nodemailer.com
- IEEE Standard 830-1998 – IEEE Recommended Practice for Software Requirements Specifications
- Data Privacy Act of 2012 (Philippines)
- Project repository documentation – `README.md`, `docs/INVITATION_EMAIL_SETUP.md`, `docs/supabase-setup-all-in-one.sql`

1.5 Overview

This document is organized into the following main sections:

- **Section 1 – Introduction.** Purpose, scope, definitions, references.
- **Section 2 – Overall Description.** Product perspective, user characteristics, constraints, assumptions.
- **Section 3 – Specific Requirements.** Interface requirements, functional requirements per module, and non-functional requirements.
- **Section 4 – Diagrams.** DFD Level 0, ERD, sequence and state diagrams.
- **Section 5 – System Architecture & Design.** High-level architecture, deployment architecture, data flow.
- **Section 6 – Technologies.** Tooling and rationale.
- **Section 7 – Testing Plan.** Test types and focus.
- **Section 8 – Expected Output.** Summary of deliverables.
- **Section 9 – Conclusion.**

---

2. Overall Description

2.1 Product perspective

Smart Document Evaluator is a **standalone web application** (not a sub-module of a larger university portal in this build). It sits between students, teachers, and Google's Gemini AI, providing a streamlined "submit → AI grade → teacher review → release" document workflow.

System context:

- **Standalone deployment:** Hosted as a single Vercel project on `https://www.smartformevaluator.com` (custom domain) with the default Vercel URL also active.
- **Google authentication only:** No local login form. Strictly Google OAuth 2.0 (PKCE) via Supabase Auth.
- **Class-list allow-list:** Only Gmail addresses on the IT332 / CS342 invitation list (plus whitelisted teacher / admin Gmails) can sign in. Non-listed accounts are immediately signed out at the access gate.
- **File storage:** Uploaded documents are stored in a private Supabase Storage bucket (`student-submissions`) with signed-URL access.
- **AI processing:** The Gemini API (or a deployer-owned proxy URL) analyzes document content against a per-doc-type rubric template.
- **No teacher-authored assignments:** Teachers do **not** create or manage assignments / tasks. The submission targets are implicit per document type. (This is the key change from earlier revisions of this SRS.)

System interfaces:

- Web browser UI for all user types (mobile-responsive)
- Google OAuth 2.0 via Supabase Auth
- Google Gemini API (REST) or a deployer-owned HTTPS proxy
- Supabase PostgreSQL for users, submissions, AI drafts, grades, audit fields

- Supabase Storage for uploaded files
- Resend API for transactional invitation emails (with Nodemailer / Gmail SMTP fallback for CLI sends)

Hardware interfaces:

- Standard web server infrastructure (managed by Vercel and Supabase; no on-premise hardware)
- No special end-user hardware; accessible from desktop, laptop, tablet, or smartphone

Software interfaces:

- **Frontend:** React 18.3.1 + TypeScript 5.6.3 + Vite 5.4.21 + Tailwind CSS 3.4.17
- **Routing:** React Router 7.15.0
- **Backend-as-a-service:** Supabase (PostgreSQL 15.x, Auth Gotrue v2, Storage)
- **Serverless function:** Vercel Functions (Node runtime) – `api/send-invitation-email.ts`
- **Transactional email:** Resend (REST API)
- **CLI / dev tooling:** Node.js 18 LTS+, Nodemailer 8.0.7, `pg` 8.20.0, `sharp` 0.34.5

Communication interfaces:

- HTTPS for all browser ↔ Supabase ↔ Gemini ↔ Vercel ↔ Resend traffic
- RESTful JSON for Supabase and Resend
- The Google Generative Language API (Gemini) is called via HTTPS / REST
- SMTP (TLS, port 465) only on the developer's machine when the Gmail fallback CLI is used

Memory constraints:

- **Client:** Minimum 2 GB RAM recommended (browser tab)
- **Server:** Managed by Supabase and Vercel — autoscaled
- **Storage:** Supabase Storage capacity is per-project plan (free tier ≥ 1 GB on Supabase free, expandable)

Operations:

- Application is publicly reachable 24/7 from the Vercel CDN
- Handles concurrent uploads and concurrent teacher review actions
- Respects Gemini API rate limits via client-side throttling; degrades to a heuristic rubric draft if the API key is missing or unreachable

2.2 User characteristics

User Type 1 — Admin

- **Role:** Maintains the class-list allow-list (`src/data/invitedStudentEmails.ts`), the email allow-list (in `api/send-invitation-email.ts`), the role whitelist (`VITE_TEACHER_EMAILS` , `VITE_ADMIN_EMAILS`), and deployment configuration.
- **Technical expertise:** High (manages source, env vars, Supabase, DNS).
- **Primary needs:** Edit allow-lists, redeploy, send / re-send invitation emails, dump database, view all submissions and grades.

User Type 2 — Teacher / faculty

- **Role:** Reviews submissions, runs the AI evaluator on demand, edits / overrides scores and feedback, requests redo, deletes invalid rows, exports CSV, releases final grades. **Teachers do not create assignments or tasks.**
- **Technical expertise:** Moderate (familiar with LMS-style review queues).
- **Primary needs:** A clear review queue, single-click access to the student file, an AI vs Teacher lane separation that prevents accidental overwrite, and a bulk-action toolbar for routine queue cleanup.

User Type 3 — Student

- **Role:** Submits documents, views status, opens AI score and Teacher score in separate dialogs once published, responds to redo requests.
- **Technical expertise:** Basic.
- **Primary needs:** Simple upload, clear submission status, friendly first-run onboarding (Eva), one-click feedback path (Rate us), and a personalized Team 14 page.

2.3 Constraints

Integration constraints

- Must use **Google OAuth 2.0 only** (no custom email-and-password form).
- Must enforce the **class-list allow-list** at the AuthContext access gate and on the serverless email endpoint.
- Must use a consistent Tailwind-based UI; no other CSS framework is mixed in.

Regulatory policies

- Comply with the **Data Privacy Act of 2012 (Philippines)**.
- Obtain consent for data processing through the institutional usage policy.
- Secure storage of student documents and grades — RLS policies enforce per-student visibility.

Technical constraints

- **File size limit:** 25 MB per document.
- **Supported formats:** PDF, DOCX, TXT, and common image formats (`.png` , `.jpg` , `.jpeg` , `.webp`). Audio / video attachments are supported only as Gemini multimodal inputs, not as student-submission types.
- **Gemini rate limits:** Default model `gemini-2.5-flash` ; per-key quotas apply (typically 60 RPM on free tier).
- **Resend free tier:** 100 emails / day, 3 000 / month. Domain must be DKIM-verified to send to non-account-owner recipients.
- Must handle API failures gracefully — heuristic fallback for Gemini, Gmail SMTP fallback for Resend.

API constraints (Gemini)

- Gemini handles entire documents (larger context than legacy NLP APIs), but is priced per-character / per-request.
- Browser-exposed `VITE_GEMINI_API_KEY` is acceptable only for development. Production deployments should use `VITE_GEMINI_EVAL_URL` (a deployer-owned HTTPS proxy) so the API key never reaches the client bundle.

Parallel operations

- Support 50+ concurrent users.
- Handle multiple simultaneous uploads.
- Manage API request queuing (client-side throttling + Resend / Gemini server-side rate limiting).

2.4 Assumptions and dependencies

Assumptions

- All target students have a personal **Gmail** account and that address is on the class-list allow-list. (The earlier `@cit.edu` requirement no longer applies — the project pivoted to Gmail because the cohort uses personal Gmail addresses.)
- Submitted documents are readable and relevant to the document type the student picks.
- Users have stable internet access for upload and download.
- The deployer maintains valid Supabase, Vercel, Resend, and (optionally) Google Cloud / AI Studio billing.

Dependencies

1. **External managed services**
   - Google OAuth 2.0 (via Supabase Auth) — critical for sign-in
   - Supabase Postgres + Auth + Storage — critical for persistence
   - Google Gemini API — required for AI grading (heuristic fallback otherwise)
   - Resend — required for invitation emails (Gmail SMTP CLI fallback otherwise)
   - Vercel — hosts the SPA and the serverless email function
   - GoDaddy (or any registrar) — owns the `smartformevaluator.com` domain; DNS delegated to Vercel nameservers
2. **Technology stack** — React 18, Vite 5, TypeScript 5, Tailwind 3, Supabase JS 2.57, react-router-dom 7.15, `mammoth` 1.12, `lucide-react` 0.344, `nodemailer` 8.0, `pg` 8.20, `sharp` 0.34.

3. **Coordination dependencies** — None. This build is **standalone**; it does not need to align with other team projects or be embedded in a wider university portal.

---

3. Specific Requirements

3.1 External interface requirements

3.1.1 Hardware interfaces

Client-side hardware

The system shall be accessible from various computing devices without requiring specialized hardware:

- **Desktop / laptop**
  - CPU: 1.5 GHz or higher
  - RAM: 2 GB minimum
  - Storage: 100 MB free space for temporary files
  - Display: 1024 × 768 resolution or higher
  - Network: 5 Mbps internet connection (10 Mbps recommended for large documents)
- **Mobile / tablet**
  - Modern evergreen browser (Chrome 110+, Safari 16+, Edge 110+, Firefox 110+)

Server-side / managed infrastructure

- **Frontend hosting** — Vercel CDN (edge), no self-managed servers
- **Database** — Supabase managed PostgreSQL (2 vCPU, 4–8 GB RAM, 100 GB SSD on the standard tier)
- **Storage** — Supabase Storage (object storage)
- **Email** — Resend (managed)
- **Domain / DNS** — Vercel-hosted DNS (registrar = GoDaddy)
- **Load balancing, SSL termination, health checks** — handled automatically by Vercel and Supabase

Network requirements

- Server bandwidth: Vercel managed; no fixed minimum on the application side
- Client internet: 5 Mbps for upload / download, < 100 ms latency for optimal experience

No special peripherals required

- Standard keyboard, mouse, touchscreen only
- Webcam / microphone not required
- No fingerprint scanners or biometric devices

Document processing hardware considerations

- Scanners / cameras: Optional for students who digitize handwritten work (not system-dependent)
- Printing: Standard printers supported via browser print functionality (`@media print` CSS implemented)

3.1.2 Software interfaces

ID	Interface	Specification
SI-1	**Google OAuth 2.0** (via Supabase Auth)	Purpose: user authentication. Protocol: OAuth 2.0 (PKCE). Data format: JWT bearer tokens. Integration: `@supabase/supabase-js` v2.57.4 client. The OAuth client is configured in Google Cloud and registered as a provider in Supabase.
SI-2	**Google Gemini API**	Purpose: AI document grading. Version: `gemini-2.5-flash` (configurable). Protocol: HTTPS REST. Authentication: API key (dev) or service-owned proxy (`VITE_GEMINI_EVAL_URL`, production). Input: document

		text + per-doc-type rubric template + multimodal attachments (PDF page images, diagrams). Output: structured JSON merged with the rubric. Fallback: heuristic rubric draft generated entirely in the browser.
SI-3	**Supabase Storage**	Purpose: store uploaded documents. Bucket: `student-submissions` (private). Access: short-lived signed URLs issued by the Supabase JS client.
SI-4	**Supabase PostgreSQL**	Tables: `public.users`, `public.assignments`, `public.submissions`, plus RLS policies; AI draft columns (`ai_draft_score`, `ai_draft_summary`) on `submissions`. Client library: `@supabase/supabase-js`. Direct DB client (CLI only): `pg` v8.20.0.
SI-5	**Resend transactional email**	Purpose: send invitation emails on first sign-in and on bulk-send. From: `Smart Docs <noreply@send.smartformevaluator.com>` (DKIM-verified). Triggered by the Vercel serverless function `/api/send-invitation-email` or the `npm run invite:*` CLIs.
SI-6	**Nodemailer + Gmail SMTP (fallback)**	Used only by `npm run invite:gmail` from the developer's machine. Host: `smtp.gmail.com:465` (TLS). Authentication: Google App Password.
SI-7	**CSV export**	Purpose: generate downloadable grade reports. No external library; CSV is composed in-browser from the reviewed submissions and downloaded as a Blob. (xlsx is no longer required.)
SI-8	**Document parsing**	Client-side text extraction for `.docx` via `mammoth` v1.12.0. PDFs and images are passed directly to Gemini as multimodal `inlineData` parts.
SI-9	**Google Forms (optional)**	Hosts the **Rate us** student-usability survey. Linked from a floating button in the student shell.

3.1.3 Communications interfaces

- **HTTPS** to Supabase REST and Auth endpoints
- **HTTPS** to `generativelanguage.googleapis.com` (or to a deployer-owned proxy) for Gemini calls
- **HTTPS** to `api.resend.com` for transactional email
- **HTTPS** to `vercel.app` / `smartformevaluator.com` for the SPA and serverless function
- **SMTPS** to `smtp.gmail.com:465` for the Gmail SMTP CLI fallback (developer machine only)
- **HTTPS/WSS** for Supabase Realtime (reserved for future use; not assumed mandatory in v3.0)

3.2 Functional requirements

> ***Important:*** *Earlier revisions of this SRS included a "Module 2 — Assignment Management (Teacher)" with FR-04 (Create Assignment) and FR-05 (Manage Assignments).* ***Those requirements are removed in v3.0.*** *Teachers in the deployed system* ***do not create or manage assignments****. The set of expected document types is fixed and the AI rubric is selected by the student-chosen* `doc type` *field at submission time.*

Module 1 — Authentication, class-list gate, and role-based access

FR-01: Google OAuth sign-in The system shall allow users to sign in **exclusively** using Google OAuth 2.0 (PKCE) through Supabase Auth. No email/password registration form shall exist.

FR-02: Class-list allow-list validation After a successful Google sign-in, the system shall check that the user's Gmail address is on **at least one** of the following:

1. The student allow-list in `src/data/invitedStudentEmails.ts` (45 entries).
2. The teacher whitelist in the `VITE_TEACHER_EMAILS` environment variable.
3. The admin whitelist in the `VITE_ADMIN_EMAILS` environment variable.

If the address is on none of them, the system shall immediately call `supabase.auth.signOut()` and display the message "*Smart Docs is for IT332 / CS342 students on the official class list only…*" on the login screen.

FR-03: Role-based access control Users shall be assigned exactly one role from `{ student, teacher, admin }`. Role determines accessible routes and queries.

- `admin` ⇒ all teacher routes + admin-only utilities
- `teacher` ⇒ teacher Dashboard, Grading, Student Submissions, Documents, Class list, Analytics, Instructions/Inbox, Settings, Team 14
- `student` ⇒ student Dashboard, Submit work, My Submissions, Tasks, Boards, Calendar, Drive, Sheets, Analytics, Team 14, Settings

FR-04: Access gate during verification While the access checks are running, the system shall display a full-screen "Verifying access" screen. The main application shell shall **not** be mounted at any point for unauthorized accounts.

Module 2 — Invitation email pipeline *(replaces the removed "Assignment Management" module)*

FR-05: First-sign-in invitation email When an invited student signs in for the first time on a given browser, the system shall POST to `/api/send-invitation-email` and send a branded HTML invitation email to the student's Gmail. The function shall use Resend and the `Smart Docs <noreply@send.smartformevaluator.com>` From address. Subsequent sign-ins shall **not** re-trigger the email (tracked in `localStorage` keyed by user id).

FR-06: Bulk invitation send A teacher / admin shall be able to bulk-send invitation emails from the command line via `npm run invite:send-all` (Resend) or `npm run invite:gmail` (Gmail SMTP fallback). The bulk-send shall support `--only=`, `--skip=`, and `--dry-run` filters and shall iterate the same class-list allow-list.

FR-07: Invitation-email allow-list The serverless email function shall enforce a server-side allow-list (mirroring `src/data/invitedStudentEmails.ts`) and reject POSTs whose `email` field is not on the list with HTTP 403.

FR-08: Audit log Successful and failed sends shall be logged to the browser console with the user id, email, and Resend response id for traceability.

Module 3 — Document submission (Student)

FR-09: View submissions and tasks Students shall see their submissions and any teacher-requested redo tasks in the **My Submissions** and **Tasks** pages.

FR-10: Submit document Students shall upload a file (PDF, DOCX, TXT, or image) and select a **document type** from `{ SRS, SDD, SPMP, STD, Other }`. The system shall validate file type and size (≤ 25 MB) before upload. The student may resubmit a new file for the same submission; the previous file URL is preserved in the row history.

FR-11: Submission confirmation and status The student shall receive an in-app confirmation of successful upload. Submission status shall be one of: `submitted`, `under_review`, `reviewed`, `redo_requested`, `final`.

Module 4 — AI evaluation (Gemini integration)

FR-12: On-demand AI evaluation When a teacher clicks **Run AI Evaluator** in the grading workspace, the system shall extract document text (via `mammoth` for `.docx`, raw for `.txt`, multimodal `inlineData` for PDF and images) and send it together with the rubric template for the document type to Gemini (or the configured proxy).

FR-13: Structured AI response The AI evaluator shall return a structured JSON payload containing: total score, per-criterion rubric scores, executive summary, verified-correct excerpts, language corrections, per-page Before → After fixes, and visual / diagram review. The system shall merge this with the rubric template and persist the result to `ai_draft_score` and `ai_draft_summary` on the submission row, setting status to `graded_ai`.

FR-14: AI failure handling If the Gemini call fails (timeout, HTTP error, malformed JSON), the system shall (a) display a clear, friendly notice in the grading modal, and (b) fall back to a deterministic heuristic rubric draft generated locally so the teacher is never blocked.

Module 5 — Teacher review queue and grading

FR-15: Review queue dashboard The teacher Dashboard and Grading pages shall show all AI-graded and pending submissions across the cohort (subject to RLS), including: student name and avatar, document type, file name, submitted-at, current status, AI score, and Teacher score.

FR-16: Open submission A teacher shall be able to open any submission file (signed URL from Supabase Storage) and see the AI grade and AI narrative side-by-side with the editable teacher fields.

FR-17: Two-lane publish — AI lane When a teacher publishes from the **Grade AI** path, the system shall persist **only** the AI-lane fields (`ai_draft_score` , `ai_draft_summary` , and the row's `status` / `feedback` as configured) and shall **not** overwrite the official `score` column.

FR-18: Two-lane publish — Teacher lane When a teacher publishes from the **Grade Teacher** path, the system shall persist **only** the official `score` and `feedback` (and `status`) and shall **not** overwrite `ai_draft_score` / `ai_draft_summary` .

FR-19: Request redo A teacher shall be able to request resubmission for any row with optional feedback. The submission status shall change to `redo_requested` and the student-side task list shall reflect it. Resubmission clears the published score per `saveReview` semantics.

FR-20: Bulk delete The class list, student submissions, grading, and submission-status pages shall expose a multi-select with a **Delete selected** action, positioned **above** the table header.

Module 6 — Final grade release and student visibility

FR-21: Final grade release When a teacher publishes a Teacher-lane grade, the submission status shall become `final` and the student shall see the grade in their portal.

FR-22: Separated student views A student shall be able to open two distinct read-only dialogs:

- **View AI score** — shows AI draft total and narrative (when present)
- **View Teacher score** — shows the official numeric grade and instructor feedback

The two dialogs shall be visually and structurally separated so AI output is never presented as the official grade.

FR-23: Print-friendly view Both students and teachers shall be able to open a print-optimized view of the evaluation; CSS `@media print` styles shall be implemented.

Module 7 — Reporting and export

FR-24: CSV grade export A teacher shall be able to export all reviewed submissions from the grading workspace to a `.csv` file with at minimum the columns: Student Name, Email, Document Type, File Name, AI Score, Teacher Score, Status, Submitted At, Reviewed At.

FR-25: Analytics dashboard The teacher Analytics page shall display: average score, submission counts by status, and AI-vs-Teacher score correlation. The student Analytics page shall display the student's own submission and score history.

Module 8 — Student onboarding and feedback

FR-26: Eva onboarding tour On the first sign-in for any new student account, the system shall run a short, dismissable guided tour ("Eva") highlighting Dashboard, Submit work, Tasks, Team 14, and Settings. The tour's "seen" flag shall be stored in `localStorage` .

FR-27: Rate us survey A floating **Rate us** button shall be visible in the bottom-right corner of every authenticated student page. Clicking it shall open the Google Forms usability survey configured at `VITE_STUDENT_RATE_US_URL` in a new tab.

FR-28: Team 14 page Both students and teachers shall be able to open a **Team 14** page that displays the project team's roster.

Module 9 — Audit logging

FR-29: Server-side audit The serverless email function shall log every invitation attempt (success or failure) with timestamp, recipient, and Resend response id.

FR-30: Client-side audit Sign-ins, grade publishes, and bulk-deletes shall be logged to the browser console with the user id and the affected submission id.

3.3 Non-functional requirements

Performance

- AI grading completes within **10 seconds** for typical documents (under load this is dominated by Gemini latency, not the app).
- First-page render is **≤ 2 seconds** on a 5 Mbps connection.
- The system shall support **50 concurrent users** initially.
- Background fetches are **throttled** so the UI does not flicker on alt-tab or route swap.

Security

- All data **encrypted in transit** (TLS 1.2+).
- Documents stored in a **private** Supabase Storage bucket, accessed only via short-lived signed URLs.
- **Row-Level Security** policies enforce per-student isolation and teacher-wide visibility.
- No storage of Google or Gemini tokens in `localStorage`; sessions handled by Supabase Auth.
- `RESEND_API_KEY`, `DATABASE_URL`, `GMAIL_APP_PASSWORD`, and any service-role keys are **never** exposed to the browser; they live only in Vercel server-side env vars or the developer's `.env` (which is gitignored).

Reliability

- **99.5% uptime** during business hours (inherited from Vercel + Supabase SLAs).
- **Graceful degradation** if Gemini is unavailable — heuristic rubric draft kicks in.
- **Automatic retry** for transient Resend / Supabase failures (up to 3 attempts client-side).
- **Sign-in stability:** Auth state changes for `TOKEN_REFRESHED` do **not** re-run the full access check so the UI does not flicker after alt-tab.

Usability

- Mobile-responsive layout (Tailwind breakpoints).
- Clear status indicators on every submission row.
- Floating **Rate us** button always reachable in the student shell.
- Friendly first-time tour ("Eva") removes the need for a separate user manual.
- WCAG 2.1 AA color contrast on the primary maroon palette.

Maintainability

- Modular React component structure (`src/components/student/`, `src/components/teacher/`, etc.).
- Environment-based configuration via `.env` + Vercel env vars; no hard-coded secrets.
- Single canonical class-list file (`src/data/invitedStudentEmails.ts`) mirrored once in `api/send-invitation-email.ts`.
- Database schema versioned as SQL files in `docs/`; bootstrap script `npm run db:apply`.
- Database dumps generated via `npm run db:dump` (uses `pg_dump`); stored in the gitignored `db-dumps/` folder.

---

4. Diagrams

> *Diagrams are maintained alongside this SRS in `docs/diagrams/` (PNG/SVG export) and as Mermaid source where possible. The textual summary of each diagram follows; replace the placeholders below with the actual diagram exports for the submitted PDF.*

4.1 DFD Level 0 (Context Diagram)

```
flowchart LR
  S[Student] -- submit file / view grade --> SD[Smart Docs Validator]
  T[Teacher] -- review queue / publish grade --> SD
  A[Admin] -- edit allow-lists / configure --> SD
```

```
  SD -- OAuth + read/write rows --> SB[(Supabase)]
  SD -- upload / signed URL --> ST[(Supabase Storage)]
  SD -- evaluate document --> GM[Google Gemini]
  SD -- POST send --> RS[Resend]
  RS -- email --> S
```

4.2 Entity-Relationship Diagram (ERD)

```
erDiagram
  USERS ||--o{ SUBMISSIONS : "submits"
  USERS {
    uuid id PK
    text email
    text full_name
    text avatar_url
    text role  "student | teacher | admin"
    timestamptz created_at
  }
  SUBMISSIONS {
    uuid id PK
    uuid user_id FK
    text doc_type "SRS|SDD|SPMP|STD|Other"
    text file_name
    text file_url
    text status "submitted|under_review|reviewed|redo_requested|final"
    numeric score
    text feedback
    numeric ai_draft_score
    jsonb ai_draft_summary
    timestamptz submitted_at
    timestamptz reviewed_at
  }
```

> *Note: `assignments` is intentionally **not** modelled as a teacher-authored entity in v3.0. If the schema retains an `assignments` table from earlier revisions, it is treated as a fixed lookup table for doc-type metadata and is **not** mutated by the UI.*

4.3 Sequence Diagram — complete workflow

```
sequenceDiagram
  actor Student
  actor Teacher
  participant App as Smart Docs SPA
  participant SB as Supabase
  participant GM as Gemini
  participant RS as Resend

  Student->>App: Google sign-in
  App->>SB: OAuth (PKCE)
  SB-->>App: JWT + profile
  App->>App: Access gate (class list)
  App->>RS: First-sign-in invitation (once)
```

```
  Student->>App: Upload file (doc type = SRS)
  App->>SB: Insert submission + Storage upload
  Teacher->>App: Open grading queue
  Teacher->>App: Run AI Evaluator
  App->>GM: prompt + doc + rubric
  GM-->>App: structured JSON
  App->>SB: Update ai_draft_*
  Teacher->>App: Override grade / publish Teacher lane
  App->>SB: Update score, feedback, status=final
  Student->>App: View Teacher score
  App-->>Student: official grade + feedback
```

4.4 State Diagram — submission lifecycle

```
stateDiagram-v2
  [*] --> submitted: Student uploads
  submitted --> under_review: Teacher opens row
  under_review --> reviewed: Teacher publishes (AI lane)
  under_review --> final: Teacher publishes (Teacher lane)
  reviewed --> final: Teacher publishes Teacher lane
  under_review --> redo_requested: Teacher requests redo
  redo_requested --> submitted: Student resubmits (clears score)
  final --> [*]
```

---

5. System Architecture & Design

5.1 High-level architecture

```
flowchart LR
  subgraph client [Browser SPA]
    UI[React + Tailwind pages + Layout]
    Auth[AuthContext + class-list gate]
    AI[Gemini eval + AIDocumentEvaluationReport]
    Notifier[InvitedStudentEmailNotifier]
  end
  subgraph vercel [Vercel]
    SPA[Static SPA]
    Fn["/api/send-invitation-email"]
  end
  subgraph supa [Supabase]
    PG[(Postgres + RLS)]
    ST[Storage bucket]
    SA[Auth - Google OAuth]
  end
  subgraph google [Google services]
    GEM[Gemini API or proxy]
    FORM[Google Forms - Rate us]
  end
  subgraph mail [Email]
    RS[Resend]
```

```
  SMTP[Gmail SMTP fallback]
end
UI --> Auth
UI --> AI
UI --> Notifier
Notifier --> Fn
Fn --> RS
Auth --> SA
AI --> GEM
UI --> FORM
SPA -.-> UI
SMTP -.fallback.-> RS
UI --> PG
UI --> ST
```

5.2 Deployment architecture

Layer	Provider	What runs there
Frontend	**Vercel** (Hobby tier)	Static SPA bundle on the Vercel global CDN
Serverless	**Vercel Functions**	`api/send-invitation-email.ts` (Node runtime)
Auth + DB + Files	**Supabase**	PostgreSQL, Auth, Storage
AI	**Google Gemini** (or deployer proxy)	Generative Language API
Email	**Resend**	Transactional sends
Domain	**GoDaddy**	Registrar only; DNS delegated to Vercel nameservers

5.3 Data flow architecture

1. User opens `https://www.smartformevaluator.com`.
2. Vercel CDN serves the static SPA.
3. The SPA calls Supabase Auth for Google OAuth (PKCE).
4. AuthContext runs the class-list / role checks; non-listed accounts are signed out.
5. Authenticated UI calls Supabase REST for `users`, `submissions`, and Storage signed URLs.
6. Teacher clicks **Run AI Evaluator** → SPA → Gemini (direct or proxy) → SPA → Supabase update.
7. Invitation send: Notifier → Vercel function → Resend → student inbox.

5.4 Module dependency diagram

- `App.tsx` ⇒ `AuthContext` ⇒ `Layout` ⇒ per-role pages
- `geminiDocumentEvaluation.ts` is consumed by both `AIDocumentEvaluationReport.tsx` and the teacher grading workspace
- `sendInvitationEmail.ts` is consumed by `InvitedStudentEmailNotifier` (UI) and shares a template (`scripts/lib/invitationEmailTemplate.mjs`) with the CLI scripts

---

6. Technologies

Layer	Technology (with version)	Reason
Frontend	**React 18.3.1**	UI framework
Frontend	**TypeScript 5.6.3**	Static typing

Build tool	**Vite 5.4.21**	Fast dev server + bundler
Styling	**Tailwind CSS 3.4.17** + PostCSS 8.5.14 + Autoprefixer 10.4.20	Utility-first CSS
Routing	**react-router-dom 7.15.0**	Client-side routing
Icons	**lucide-react 0.344.0**	Icon set
Doc parsing	**mammoth 1.12.0**	`.docx` text extraction
Backend-as-a-service	**Supabase JS 2.57.4**, Postgres 15.x, Auth (Gotrue v2), Storage	Database, auth, file storage
Serverless email	**Vercel Functions** (Node runtime) + **Resend** REST API	Transactional invitation emails
Email fallback	**Nodemailer 8.0.7** + Gmail SMTP	CLI bulk sender when Resend domain is not yet verified
AI / ML	**Google Gemini API** (`gemini-2.5-flash`, configurable)	Document understanding + rubric scoring
Hosting	**Vercel** (Hobby tier)	Auto-deploys from GitHub `main`; managed CDN + SSL
Domain / DNS	**GoDaddy → Vercel nameservers**	Custom domain `smartformevaluator.com`
Survey	**Google Forms**	Optional Rate-us survey
Tooling (dev)	ESLint 9.12.0, typescript-eslint 8.8.1, eslint-plugin-react-hooks 5.1.0-rc, eslint-plugin-react-refresh 0.4.12	Linting
Tooling (dev)	**pg 8.20.0**	Postgres client for `npm run db:apply`, `npm run db:dump`
Tooling (dev)	**sharp 0.34.5**	Mascot PNG matte removal
Runtime	**Node.js 18 LTS** (tested up to 22.x)	Used by Vite, CLIs, and the Vercel function
Package manager	**npm 10+**	Lockfile committed

> ***What is NOT in this stack (vs. earlier revisions):***
>
> - ***No Node.js + Express backend.*** *All server logic is either Supabase or one Vercel function.*
> - ***No MySQL / Sequelize.*** *The database is Supabase PostgreSQL.*
> - ***No Passport.js.*** *Google OAuth runs through Supabase Auth.*
> - ***No Socket.IO.*** *Realtime is not used in v3.0 (Supabase Realtime is reserved for future use).*
> - ***No Google Cloud Run / App Engine.*** *Hosting is Vercel.*

---

7. Testing Plan

Test type	Focus
Unit testing	Individual React components, helper libraries (`geminiDocumentEvaluation`, `sendInvitationEmail`, `classRosterCache`), serverless function input validation

Integration testing	Google OAuth sign-in via Supabase, class-list gate, Gemini integration with both API key and proxy URL paths
File upload testing	PDF, DOCX, TXT, PNG / JPG, edge cases (0-byte, 25 MB, password-protected, scanned PDFs)
AI accuracy testing	Compare AI rubric scores vs. teacher-published scores across a sample set; tune prompt template
Email pipeline testing	`npm run invite:test`, `npm run invite:send-all --dry-run`, end-to-end Resend send, Gmail SMTP fallback
Access-gate testing	Sign in with a listed Gmail (must succeed), unlisted Gmail (must show steady error, never the app shell), expired Supabase session (must redirect to login)
Load testing	50 concurrent users uploading and viewing the grading queue
Usability testing	Eva onboarding flow for first-time students, Rate us survey completion rate, teacher review-queue clarity
Security testing	Class-list bypass attempts, RLS policy enforcement (student cannot read another student's submission), Resend allow-list rejection, key-exposure scan
Regression testing	After any change to `AuthContext`, re-test the gate; after any change to `geminiDocumentEvaluation`, re-test the heuristic fallback path

---

8. Expected Output

A functional web application that:

1. Authenticates users **exclusively via Google OAuth** and enforces a class-list allow-list at the access gate.
2. **Does not** ask teachers to create assignments or tasks — submissions are categorized by a fixed `doc type` chosen by the student.
3. Enables students to upload documents (PDF, DOCX, TXT, image) and respond to redo requests.
4. Automatically grades submissions using **Google Gemini** against a per-doc-type rubric.
5. Provides teachers with a **review queue** with **AI lane / Teacher lane isolation** so neither publish overwrites the other.
6. Releases final grades to students only after the teacher publishes the Teacher lane.
7. Exports grades to **CSV** for record-keeping and supports browser print views.
8. Sends **branded invitation emails** via Resend on first sign-in and on bulk-send.
9. Is deployed end-to-end on **Vercel + Supabase + Resend + GoDaddy DNS** at `https://www.smartformevaluator.com`.

---

9. Conclusion

Smart Document Evaluator transforms the traditional manual grading workflow into an efficient, AI-assisted, **teacher-in-the-loop** process for the IT332 / CS342 cohort. By leveraging **Google Gemini** to draft rubric scores, executive summaries, and per-page Before / After fixes, the teacher can focus on reviewing and refining feedback rather than grading from scratch.

The deployed v3.0 build deliberately removes the teacher-creates-assignments scope of earlier revisions: the cohort's expected submissions are already known per document type, so the AI rubric is selected automatically and teachers only review. The strict Google OAuth + class-list allow-list aligns with the cohort's privacy requirements, the **two-lane publish** model preserves academic integrity, and the **standalone deployment** on Vercel + Supabase removes the integration overhead of the earlier multi-team vision. The architecture, while small, is modular enough to be re-introduced as a sub-module of a larger university portal in a future revision without rewriting the core grading flow.

This solution directly addresses the professor's requirements for **document processing, AI scoring, teacher review loop, and reporting**, and is the version currently running in production at https://www.smartformevaluator.com.

End of SRS document v3.0 — May 13, 2026.

CEBU INSTITUTE OF TECHNOLOGY – UNIVERSITY

COLLEGE OF COMPUTER STUDIES

Software Design Description (SDD)

for

Smart Document Evaluator (Smart Docs Validator) with AI Integration

Prepared by:
Alexandrei Nash Dinapo
Jeffer Azcona
Jushua Peter Te
Ryan Bebiro

Date: May 14, 2026
Version: 3.0.1 — revised to match the deployed system (supersedes SDD v2.0 / PDF *SDD_SMARTDOCUMENTGRADER*)

Change History

Version	Date	Description	Author
0.1	Dec 17, 2025	Initial SDD draft (legacy stack narrative)	Alexandrei Nash Dinapo
1.0	Dec 18, 2025	First complete SDD	Alexandrei Nash Dinapo
2.0	Feb 21, 2026	SDD update (still referenced Express / MySQL in places)	Alexandrei Nash Dinapo
3.0	**May 14, 2026**	**Full revision:** Replaced obsolete Node/Express/Sequelize/MySQL design with the **actual** Vite + React + TypeScript SPA, **Supabase** (Postgres + Auth + Storage), **Vercel** serverless (`/api/*`), **Resend** + SMTP invitation pipeline, **same-origin Gemini proxy**, multimodal grading (`shared/*`), teacher review (`ReviewQueue`), student quick submit hardening, and deployment on `vercel.app` + custom domain. Aligns with **SRS v2 baseline** and follow-on **SRS v3.1** (`docs/SRS-Smart-Document-Evaluator-v3.md`).	Project team
3.0.1	**May 14, 2026**	**Course Tasks subsystem:** `TeacherCourseAssignments` route, handout upload (`uploadAssignmentHandout`), optional `handout_url` / `handout_file_name` columns, student `openTask` query on Submit Work, `StudentDeadlineReminders`, workspace filter for General Submission title, bulk/single delete. See §3.8.	Project team

---

Table of Contents

---

1. Introduction

1.1 Purpose

This Software Design Description (SDD) specifies **how** the **Smart Document Evaluator** (UI brand: **Smart Docs Validator**) is structured and implemented so that it satisfies the requirements in `docs/SRS-Smart-Document-Evaluator-v3.md` (v3.1 amendments) and the detailed baseline in `docs/SRS-Smart-Document-Evaluator-v2.md`.

It replaces outdated material in earlier SDD revisions (including the legacy **Express + Sequelize + MySQL** narrative and duplicated "Smart Form Validator" boilerplate found in the PDF export *SDD_SMARTDOCUMENTGRADER.pdf*) with the **as-built** architecture: a **Vite + React** SPA, **Supabase** backend, **Vercel** hosting and serverless functions, and **Google Gemini** accessed via a **server-side proxy** in production.

Primary audiences: developers maintaining the repo, deployers configuring Vercel/Supabase, course staff validating design vs. implementation, and academic reviewers.

1.2 Scope

This SDD covers:

- Front-end structure (pages, shared components, client libraries).
- Back-end **as implemented**: Supabase (database, auth, storage policies) and **Vercel Node-style** API routes under `api/`.
- AI grading pipeline (`shared/geminiDocumentEvaluation.ts` and related modules, `api/gemini-evaluate.ts`).
- Email invitation design (`api/send-invitation-email.ts`, `scripts/lib/invitationEmailTemplate.mjs`).
- Deployment, configuration (environment variables), and security boundaries.

Out of scope: formal UML for every React component, proprietary Google/Supabase internal implementation details, and institution-wide LMS integration beyond hyperlinks and CSV export.

1.3 Definitions, Acronyms, and Abbreviations

Term	Definition
SPA	Single-page application served from `dist/` after `vite build`.
Supabase	Managed Postgres + Auth + Storage; app uses `@supabase/supabase-js`.
RLS	Row-Level Security policies on Postgres tables.
Vercel Function	Serverless handler in `api/*.ts` exposed as HTTPS same-origin routes.
Gemini proxy	`POST /api/gemini-evaluate` — runs Gemini with `GEMINI_API_KEY` on the server; browser never needs the key in production.
Eval payload	JSON body `{ docType, content, template, attachments?, model? }` consumed by `runGeminiBackedEvaluation`.
Multimodal attachment	Base64 `inlineData` (PDF pages, images, etc.) clamped for Vercel body limits (`shared/geminiProxyPayload.ts`).
AI lane / Teacher lane	AI draft fields (`ai_draft_score`, `ai_draft_summary`) vs. official `score` / `feedback`.
Resend	Transactional email API used by `send-invitation-email`.

1.4 References

- **SRS (requirements):** `docs/SRS-Smart-Document-Evaluator-v3.md` (v3.1 amendments) and `docs/SRS-Smart-Document-Evaluator-v2.md` (detailed FR baseline).
- **Repository:** `README.md`, `.env.example`, `docs/INVITATION_EMAIL_SETUP.md`, `docs/supabase-setup-all-in-one.sql` (and related SQL under `docs/`).
- **Prior SDD (superseded technical stack):** `SDD_SMARTDOCUMENTGRADER.pdf` (v2.0) — retained for change history only.
- **External:** Gemini API, Supabase Docs, Vercel Docs, Resend Docs.

---

2. System Context and Architectural Design

2.1 High-Level Context

```
flowchart LR
  subgraph clients["Clients"]
    T[Teacher browser]
    S[Student browser]
  end

  subgraph vercel["Vercel-hosted SPA + API"]
    SPA[React SPA / Vite]
    APIg["/api/gemini-evaluate"]
    APIe["/api/send-invitation-email"]
  end

  subgraph google["Google"]
    OAuth[Google OAuth]
```

```
  Gemini[Gemini API]
end

subgraph supa["Supabase"]
  Auth[Auth service]
  DB[(Postgres)]
  Stor[Storage buckets]
end

subgraph email["Email"]
  Resend[Resend API]
end

S --> SPA
T --> SPA
SPA --> Auth
SPA --> DB
SPA --> Stor
SPA --> APIg --> Gemini
SPA --> APIe --> Resend
Auth --> OAuth
```

2.2 Logical Layered Architecture

Layer	Responsibility	Primary location
Presentation	Routes, modals, forms, grading UI	`src/pages/**`, `src/components/**`
Application / client services	Supabase calls, Gemini runtime resolution, submission URL resolution	`src/lib/**`, `src/context/AuthContext.tsx`
Domain / AI (shared)	Prompting, JSON parse/repair, rubric merge, persistence helpers for AI text	`shared/*.ts`
Serverless API	CORS, secrets, orchestration	`api/gemini-evaluate.ts`, `api/send-invitation-email.ts`
Data	Tables, RLS, Storage	Supabase project (see `docs/*.sql`)

2.3 Technology Stack

Concern	Technology
UI	React 18, TypeScript, Vite, Tailwind-style utility classes, `react-router-dom` v7
Auth	Supabase Auth (Google provider), PKCE
Database	PostgreSQL via Supabase
File storage	Supabase Storage (bucket configurable via `VITE_SUBMISSION_STORAGE_BUCKET`) or inline data URL fallback under size caps
AI	Google Gemini (`generativelanguage.googleapis.com`), model list / fallbacks in `shared/geminiDocumentEvaluation.ts`
Hosting	Vercel (`vercel.json` rewrites SPA; `api/*` as functions)

Email	Resend (HTTP) from `api/send-invitation-email.ts`; optional Gmail SMTP in `scripts/*`

2.4 Deployment Architecture

- **Build:** `npm run build` → static assets in `dist/`.
- **Routing:** Vercel rewrite sends non-API paths to `index.html` for client routing (`vercel.json`).
- **Functions:** `api/gemini-evaluate.ts` bundles `shared/**` (`includeFiles`) and may run up to **60s** (`maxDuration`).
- **Secrets:** `GEMINI_API_KEY` (and optional `VITE_GEMINI_MODEL`, `GEMINI_PROXY_EXTRA_ORIGINS`, `SMARTDOCS_APP_URL`, Resend keys, etc.) live in **Vercel Environment Variables** for the project that owns the deployment URL.
- **Custom domain:** e.g. `www.smartformevaluator.com` — must be listed in Gemini proxy origin allow-list alongside `*.vercel.app` and localhost (`api/gemini-evaluate.ts`).

2.5 Key Design Principles

1. **Teacher-in-the-loop:** AI produces a **draft**; publish actions write official fields.
2. **Secrets off the client in production:** Prefer `/api/gemini-evaluate` (`src/lib/geminiEvalClient.ts`).
3. **Shared evaluation logic:** Same TypeScript module runs in the browser (dev / direct key) and on Vercel (server key), avoiding drift.
4. **Defense in depth on uploads:** Client-side in-flight guard + `preventDefault` on submit forms (`src/pages/student/Assignments.tsx`) to avoid duplicate rows.
5. **Graceful AI degradation:** Tiered prompts and JSON repair in `shared/geminiDocumentEvaluation.ts` reduce fallback to heuristic scoring.

---

3. Detailed Design by Subsystem

3.1 Authentication, Authorization, and Class-List Gate

Design intent: Only invited cohort accounts and whitelisted staff use the app.

Implementation:

- `src/context/AuthContext.tsx` — session lifecycle, profile row in `public.users`, role (`student` | `teacher` | `admin`).
- Class-list gate — `src/data/invitedStudentEmails.ts` (and related checks) per SRS; sign-in blocked for unknown Gmail addresses.
- Route guards — teacher vs. student routes under `src/pages/**` with layout wrappers as applicable.

Sequence (simplified OAuth via Supabase):

```
sequenceDiagram
  participant U as User
  participant SPA as React SPA
  participant SA as Supabase Auth
  participant G as Google OAuth

  U->>SPA: Click Sign in with Google
  SPA->>SA: signInWithOAuth (PKCE)
  SA->>G: redirect / consent
  G->>SA: authorization code
  SA->>SPA: session JWT + user id
  SPA->>SPA: upsert profile / role check
```

3.2 Student Submission and Storage

Modules: `src/pages/student/Assignments.tsx`, `src/pages/student/Submissions.tsx`, `src/lib/submissionStorage.ts`, `src/lib/localSubmissionSync.ts` (when applicable).

Upload pipeline:

1. Student selects file + metadata (Quick submit: required **document type** label `submission_doc_type`).
2. Client optionally extracts plain text for small text-like files.
3. `resolveStudentSubmissionFileUrl` uploads to Storage **or** embeds as `data:` URL under configured max size.
4. Insert row into `submissions` (or `submission` singular variant) with `file_name`, `file_url`, `status`, optional `submission_doc_type`.

Quick submit / Turn in UI (v3.0 behavior):

- `submitInFlightRef` + `saving` guard prevents double POST before React re-renders disabled buttons.
- File input `ref` + clear (**X**) resets DOM value so the same file can be re-chosen after a mistake.

```
sequenceDiagram
  participant S as Student
  participant SPA as Assignments.tsx
  participant ST as Storage API
  participant DB as Postgres

  S->>SPA: Choose file + doc type + Send
  SPA->>SPA: preventDefault + in-flight lock
  SPA->>ST: upload (if bucket configured)
  ST-->>SPA: public URL
  SPA->>DB: insert submission
  DB-->>SPA: ok
  SPA->>SPA: unlock + refresh list
```

3.3 AI Evaluation (Gemini)

Modules:

- `shared/geminiDocumentEvaluation.ts` — prompts (`buildPrompt`, fallbacks), `runGeminiBackedEvaluation`, JSON extraction/repair, normalization, executive summary enrichment.
- `shared/geminiAttachments.ts`, `shared/geminiProxyPayload.ts`, `shared/geminiInlineTypes.ts` — attachment typing and Vercel-safe payload clamping.
- `src/lib/geminiEvalClient.ts` — `resolveGeminiEvalRuntime()` (production → same-origin proxy first).
- `api/gemini-evaluate.ts` — validates Origin, reads `GEMINI_API_KEY`, calls `runGeminiBackedEvaluation` with `evalUrl: null` (direct server key path).

Processing flow:

```
sequenceDiagram
  participant T as Teacher browser
  participant API as /api/gemini-evaluate
  participant EV as runGeminiBackedEvaluation
  participant GM as Gemini REST

  T->>API: POST JSON (template + text + attachments)
  API->>EV: server-side call
  EV->>GM: Tier 1 medium prompt
  alt parse OK
    GM-->>EV: JSON
  else truncate / parse fail
```

```
    EV->>GM: Tier 2 minimal / Tier 3 ultra-minimal
  end
  EV-->>API: criteria + summary + extras
  API-->>T: 200 JSON
```

Design notes:

- **GET** health on same route returns `{ ok, serverKeyConfigured }` without leaking the key.
- **POST** body size managed by clamping inline attachments to stay under Vercel limits.
- **UI alignment:** `src/pages/teacher/ReviewQueue.tsx` probes readiness, passes structured props into `AIDocumentEvaluationReport`, and persists AI draft blobs per SRS.

3.4 Teacher Review Queue and Publishing

Module: `src/pages/teacher/ReviewQueue.tsx` (large stateful screen — grading modal, AI-only vs teacher lane, sessionStorage locks for AI-only flow).

Responsibilities:

- Load submission text + binary attachments for Gemini (`loadSubmissionAttachmentsForGemini` pattern per codebase).
- Run AI evaluation → merge into rubric UI → optional freeze (`sessionStorage`) for AI-only grading mode.
- Publish: writes `score`, `feedback`, `status`, and AI draft columns per schema.

3.5 Student Grade and Report Views

Modules: `src/pages/student/Submissions.tsx`, `src/components/AIDocumentEvaluationReport.tsx`.

Design: Read-only presentation of published teacher data + parsed AI extras from stored summary format (`parsePersistedAiDraftSummary` / append tail conventions in `shared/geminiDocumentEvaluation.ts`).

3.6 Invitation Email Pipeline

Modules:

- `api/send-invitation-email.ts` — Resend HTTP API, CORS, HTML + text bodies, default `SMARTDOCS_APP_URL` → **`…/login`** for "Open Smart Docs".
- `scripts/lib/invitationEmailTemplate.mjs` + `scripts/send-invitation-email-*.mjs` — CLI parity with server template.

Sequence:

```
sequenceDiagram
  participant SPA as SPA (first sign-in hook)
  participant API as /api/send-invitation-email
  participant R as Resend

  SPA->>API: POST invite payload
  API->>R: send transactional mail
  R-->>API: 200 / message id
  API-->>SPA: success (no secrets)
```

3.7 Analytics, Export, and Supporting Utilities

- **Teacher analytics:** `src/pages/teacher/Analytics.tsx` — aggregates from Supabase queries.
- **CSV export:** Implemented in teacher flows per SRS (see `ReviewQueue` / export helpers in repo).
- **Local fallback:** `localSubmissionSync` and local storage keys for offline-first dev scenarios (see code for exact behavior).

3.8 Course Tasks (Teacher Publishing)

Purpose: Teachers publish cohort-visible tasks (assignments) with optional due date and optional handout file; students consume them on **Tasks** and deep-link into **Submit Work**.

Key modules:

- `src/pages/teacher/TeacherCourseAssignments.tsx` — list teacher-owned rows from `assignments` (or `assignment`); create modal; close/re-open; single and bulk delete; handout file picker with clear control; custom due-date popover (date + time + OK).
- `src/lib/submissionStorage.ts` — `uploadAssignmentHandout(file, teacherId)` writes to the submissions bucket under `handouts/{teacherId}/…`.
- `src/App.tsx` — route `/course-tasks` (teacher-only).
- `src/components/Sidebar.tsx` — nav label **Course Tasks**.

Student integration:

- `src/lib/studentWorkspaceData.ts` — `safeFetchAssignments` selects extended columns when present; filters out **General Submission** title before exposing assignments to `useStudentWorkspace`.
- `src/pages/student/Tasks.tsx` — Turn in links to `/assignments?openTask={id}`; handout link when URL present.
- `src/pages/student/Assignments.tsx` — reads `openTask` search param, opens turn-in modal, strips param after consume.
- `src/components/student/StudentDeadlineReminders.tsx` — in-app alerts + optional `Notification` API.

Data: SQL migrations `docs/supabase-assignments-handout.sql`, `docs/supabase-assignments-document-type-add-std.sql`; core table definitions in `docs/supabase-assignments-submissions-core.sql` (includes `assignments_delete_own` RLS).

---

4. Data Design

Authoritative schema: apply SQL under `docs/` (e.g. `supabase-setup-all-in-one.sql`, `supabase-submissions-submission-doc-type.sql`).

Core entities (logical):

- **users** — mirrors auth user; `role` drives UI routing and RLS expectations.
- **assignments** — course tasks (system may auto-ensure a “general submission” bucket).
- **submissions** — `student_id`, `assignment_id`, `file_name`, `file_url`, `status`, `score`, `feedback`, `ai_draft_score`, `ai_draft_summary`, optional `submission_doc_type`.

Submission status state machine (simplified):

```
stateDiagram-v2
  [*] --> submitted
  submitted --> under_review
  under_review --> reviewed
  under_review --> redo_requested
  redo_requested --> submitted
```

---

5. Interface and Integration Design

5.1 External APIs

API	Direction	Purpose
Supabase REST / Realtime	Client → Supabase	CRUD on tables, auth session

Supabase Storage	Client → Supabase	Binary upload
Gemini `generateContent`	Server (`api/gemini-evaluate`) → Google	Grading
Resend	Server → Resend	Invitation email

5.2 Internal "API" (Same Origin)

Route	Method	Body / response
`/api/gemini-evaluate`	GET	Health / key configured flag
`/api/gemini-evaluate`	POST	Eval request JSON → grading JSON
`/api/send-invitation-email`	POST	Invite payload → send result

CORS: Gemini function validates `Origin` / `Referer` against allow-list; invitation function follows pattern documented in `api/send-invitation-email.ts`.

---

6. Security and Privacy Design

- **OAuth tokens:** Managed by Supabase client; not stored in custom cookies beyond SDK defaults.
- **RLS:** Enforce student ownership and teacher read policies in SQL (`docs/supabase-rls-*.sql`).
- **API keys:** `GEMINI_API_KEY` and Resend keys only on server env; students never receive Gemini secrets in production path.
- **Attachment handling:** Base64 in JSON over HTTPS; teachers should treat AI output as advisory (SRS non-functional).
- **Privacy (Philippines DPA):** Align collection and retention with institutional policy; data minimization in prompts (judge from submission only).

---

7. Error Handling, Resilience, and Operational Concerns

Risk	Mitigation
Gemini timeout / 429	Model candidate list + retryable detection in shared Gemini client code paths
Truncated JSON	`repairTruncatedJson` + tiered prompts
Oversized POST to Vercel	`clampInlineAttachmentsForVercelProxy`
Missing DB columns	Feature detection + user-facing notices (e.g. `submission_doc_type`)
Double submit	`submitInFlightRef` + disabled buttons + `preventDefault`
Wrong deployment URL in email	`SMARTDOCS_APP_URL` env + `/login` suffix in templates

---

8. Traceability to SRS

SRS area (v3)	SDD section
Class-list gate & roles	§3.1
Student submit / doc type	§3.2
Gemini grading & teacher review	§3.3, §3.4

Student-visible reports	§3.5
Invitation email	§3.6
CSV / analytics	§3.7
NFR: security, deployability	§2.4, §5, §6, §7

Document Control

- **Master copy (Markdown):** `docs/SDD-Smart-Document-Evaluator-v3.md` in repository root project.
- **Export:** For submission to CIT-U, print this file to PDF from your editor or Pandoc; update the **Change History** table when the version increments.

End of SDD v3.0

CEBU INSTITUTE OF TECHNOLOGY – UNIVERSITY

COLLEGE OF COMPUTER STUDIES

Software Project Management Plan (SPMP)

for

Smart Document Evaluator (Smart Docs Validator)

Prepared by: Alexandrei Nash Dinapo · Jeffer Azcona · Jushua Peter Te · Ryan Bebiro

Document version: 3.0
Date: May 14, 2026

Note on templates: Structured to follow a typical **SPMP** table of contents (IEEE 1058 / course-style project management). Source Word template `SPMP TEMPLATE.doc` was not machine-readable; content reflects the **as-built** repository and deployment.

Change History

Version	Date	Description
1.0	May 14, 2026	Initial SPMP for final deliverable package.

Table of Contents

---

1. Overview

1.1 Project summary

Smart Document Evaluator (branded **Smart Docs Validator**) is a capstone / course-aligned web system for document submission, AI-assisted rubric evaluation (Google Gemini), teacher review, class roster management, and (as of v3.1) **Course Tasks** with optional handouts and deadline reminders. The production stack is **React + TypeScript + Vite**, **Supabase** (Postgres, Auth, Storage), **Vercel** (static SPA + serverless `api/*`), **Resend** (invitation email), and optional **Google Forms** for usability feedback.

1.2 Objectives

- Deliver a maintainable SPA with secure role separation and RLS-backed data access.
- Provide reliable submission and grading flows with AI + teacher lanes.
- Document requirements (SRS), design (SDD), tests (STD), and management (this SPMP) for academic sign-off.
- Support reproducible deployment (`README.md` , `docs/supabase-*.sql` , environment variables).

1.3 Constraints

- Course timelines and adviser review gates.
- Third-party quotas (Gemini, Resend, Supabase free tiers).
- OAuth-only authentication; no custom password store.

---

2. Project deliverables

Deliverable	Location / artifact
Source code	GitHub repository (`main` branch); zip export per course instructions.
SRS	`docs/SRS-Smart-Document-Evaluator-v3.md` (+ baseline detail in `docs/SRS-Smart-Document-Evaluator-v2.md`).
SDD	`docs/SDD-Smart-Document-Evaluator-v3.md`.
SPMP	`docs/SPMP-Smart-Document-Evaluator-v3.md` (this file).
STD	`docs/STD-Smart-Document-Evaluator-v3.md`.
ReadMe / deployment	Root `README.md` (export to **PDF** separately for submission binder).
Database	`npm run db:dump` → `db-dumps/` (gitignored; attach per instructions).
Usability instrument	`docs/SoftwareUsabilitySurvey-StudentRateUs.md` + generated Google Form.
Research paper	*(separate ACM-format document — not stored in this repo by default.)*

---

3. Organization and roles

Role	Responsibility
Team lead / integrator	Git workflow, Vercel/Supabase env alignment, merge readiness.
Frontend	React pages, AuthContext, student/teacher UX, Tailwind styling.
Backend / data	Supabase SQL, RLS policies, Storage policies, `api/*` serverless.
AI / grading	Gemini payload shaping, heuristic fallback, ReviewQueue integration.
QA	Manual test passes per STD; regression before demos.
Adviser	Scope approval, final presentation go-signal (record sign-off externally).

---

4. Management processes

4.1 Communication

- Primary: course meetings, adviser consults, team chat (external).
- Technical: GitHub issues/PRs (if used), commit messages, `README.md` updates.

4.2 Change control

- Functional changes require SRS amendment (v3.1 table) + SDD update + STD test updates.
- Schema changes: new file under `docs/supabase-*.sql` + note in `README.md`.

4.3 Progress monitoring

- Milestones tied to course calendar (prototype → integration → hardening → documentation → presentation).
- Build health: `npm run typecheck`, `npm run lint`, `npm run build` before merges.

---

5. Work breakdown and milestones

Phase	Work packages	Exit criteria
Requirements	SRS baseline + v3.1 amendments	Adviser review of scope
Design	SDD context, subsystems, data design	Matches deployed architecture
Implementation	SPA, Supabase, APIs, AI pipeline	Core flows demoable
Hardening	Auth gate, double-submit guards, RLS fixes	No P1 bugs open
Verification	STD execution + usability survey	Evidence in binder
Documentation	SPMP, STD, ReadMe PDF, zip	Checklist complete
Closure	Presentation, peer eval, research paper	Course submission

---

6. Schedule and dependencies

Dependencies: Supabase project availability; Google Cloud OAuth client; Vercel project; optional Resend domain verification; Gemini API key for live AI demos.

Critical path (typical): Auth + roster gate → submissions + storage → teacher queue → AI integration → polish + docs.

(Insert Gantt or week table here per your course template — placeholder.)

---

7. Risk management

Risk	Probability	Impact	Mitigation
Gemini outage / quota	Med	High	Heuristic fallback; show notice; record in STD
Resend / DNS failure	Low	Med	Document Gmail CLI fallback; test `invite:test`
RLS misconfiguration	Med	High	Apply `docs/supabase-fix-users-rls-recursion.sql`; test student/teacher isolation
PII in committed dumps	Med	High	Keep `db-dumps/` gitignored; share dumps out-of-band only
Scope creep	Med	Med	Amend SRS §3.2; adviser approval

---

8. Quality assurance

- **Static:** TypeScript strict project (`npm run typecheck`), ESLint.
- **Dynamic:** Manual scenarios in STD; optional Cursor/browser MCP for UI smoke.

- **Usability:** Google Form survey; collate Likert + comments into findings appendix.
- **Security:** No secrets in client bundle for production Gemini; verify `vercel.json` rewrites.

---

9. Configuration management

- **VCS:** Git; `main` protected for production deploys.
- **Branches:** Feature branches → PR → merge (team convention).
- **Secrets:** Vercel + local `.env` (never commit `.env`).
- **Lockfile:** `package-lock.json` committed for reproducible installs.

---

10. Deployment and operations

- **Frontend:** `npm run build` → `dist/` on Vercel; see `README.md` §Deployment.
- **Backend:** Supabase SQL editor for migrations; Storage bucket policies.
- **Monitoring:** Vercel build logs; Supabase Auth logs; browser console for client errors.
- **Backup:** `npm run db:dump` before major schema changes.

---

11. References

- `docs/SRS-Smart-Document-Evaluator-v3.md`
- `docs/SDD-Smart-Document-Evaluator-v3.md`
- `docs/STD-Smart-Document-Evaluator-v3.md`
- `docs/PRESENTATION-READINESS-AND-DELIVERABLES.md`
- IEEE Std 1058-1998 (SPMP standard, as reference reading).

End of SPMP v3.0

CEBU INSTITUTE OF TECHNOLOGY – UNIVERSITY

COLLEGE OF COMPUTER STUDIES

Software Test Document (STD)

for

Smart Document Evaluator (Smart Docs Validator)

Prepared by: Alexandrei Nash Dinapo · Jeffer Azcona · Jushua Peter Te · Ryan Bebiro

Document version: 3.0
Date: May 14, 2026

Note on templates: Follows a conventional **STD** layout (test plan, environment, cases, traceability). Source Word template `STD TEMPLATE.doc` was not machine-readable; cases below are **manual** and map to **SRS v3.1** (`docs/SRS-Smart-Document-Evaluator-v3.md`) and baseline **SRS v2** FRs.

Change History

Version	Date	Description
1.0	May 14, 2026	Initial STD for deliverable package.

Table of Contents

---

1. Introduction

1.1 Purpose

Define verification approach for Smart Docs Validator: authentication, submissions, AI grading, teacher publishing, course tasks, reminders, email, and deployment configuration.

1.2 Scope

In scope: black-box and gray-box **manual** tests executable by a student tester with teacher and student Google accounts. **Out of scope:** automated E2E CI (not required by this document); load testing beyond informal multi-user check.

1.3 Definitions

Term	Meaning
SUT	System under test — deployed Vercel URL or `localhost:5173`.
Pass	Expected result observed with no blocking defect.

---

2. Test plan

2.1 Objectives

- Confirm FR coverage for critical paths (auth, submit, grade, course tasks).
- Record evidence (screenshots, dates, tester initials) for presentation binder.

2.2 Entry criteria

- Build succeeds (`npm run build`).
- Supabase schema applied; test accounts configured per `README.md` .

2.3 Exit criteria

- All **Priority P1** cases in §4 pass on staging or production.
- Known P2 issues listed with workaround.

2.4 Responsibilities

Activity	Owner
Execute tests	QA / any team member
Sign-off	Team lead + adviser (external record)

---

3. Test environment

Item	Specification
Browser	Chrome (current), Edge, or Safari
OS	Windows 11 / macOS
URLs	Production `https://www.smartformevaluator.com` and/or Vercel preview
Accounts	At least one **student** on class list; one **teacher** in `VITE_TEACHER_EMAILS`; optional **admin**
Tools	DevTools Network tab; Supabase dashboard (read-only)

---

4. Feature test cases

Columns: ID · Objective · Preconditions · Steps · Expected

4.1 Authentication and access gate

ID	Objective	Preconditions	Steps	Expected

TC-AUTH-01	Student allow-list	Gmail on roster	Sign in with roster student	Lands on student Dashboard
TC-AUTH-02	Block non-roster	Gmail not on roster	Sign in	Access denied message; signed out
TC-AUTH-03	Teacher role	Email in `VITE_TEACHER_EMAILS`	Sign in	Teacher routes visible (e.g. Grades, Class List)

4.2 Submission and storage

ID	Objective	Preconditions	Steps	Expected
TC-SUB-01	Quick submit	Student session	Submit Work → quick upload valid PDF	Row appears in Submission Status
TC-SUB-02	Turn in for task	Active course task	Tasks → Turn in → upload	Submission linked; `openTask` cleared from URL
TC-SUB-03	File limit	Large file	Attempt upload > limit	Friendly validation (per app behavior)

4.3 AI evaluation and teacher publish

ID	Objective	Preconditions	Steps	Expected
TC-AI-01	Run AI evaluator	Teacher; submission pending	Open grading modal → run AI	AI draft fields populated or heuristic notice
TC-AI-02	Publish teacher lane	AI exists	Edit score/feedback → publish teacher	Student sees teacher score

4.4 Course tasks (teacher)

ID	Objective	Preconditions	Steps	Expected
TC-CT-01	Create task	Teacher	Course Tasks → New → title + OK due date → publish	Task listed
TC-CT-02	Handout optional	Teacher	Attach file → publish; repeat without file	Storage URL when columns exist
TC-CT-03	Close / re-open	Task exists	Close then Re-open	Status badges update
TC-CT-04	Delete one	Task exists	Delete → confirm	Row removed
TC-CT-05	Bulk delete	Multiple tasks	Select two → Delete selected → confirm	Both removed
TC-CT-06	STD type	Teacher	Create with document type STD	Saves without DB check error (after migration)

4.5 Student workspace and reminders

ID	Objective	Preconditions	Steps	Expected
TC-ST-01	General hidden	General bucket exists	Open Tasks	No "General Submission" row
TC-ST-02	Deadline UI	Task due soon	Open Dashboard/Tasks	Reminder banner if within window
TC-ST-03	Handout link	Task with handout	Student Tasks / Submit Work	Download link works

4.6 Invitation email

ID	Objective	Preconditions	Steps	Expected
TC-EM-01	First sign-in mail	Resend configured	New student first login	Single branded email (check inbox)

---

5. Non-functional tests

ID	Objective	Method	Expected
TC-NF-01	HTTPS only	Observe URL bar	No mixed-content warnings for main flows
TC-NF-02	No Gemini key in bundle	View page source / network	Production calls `/api/gemini-evaluate`, not raw key in JS
TC-NF-03	RLS isolation	Two student accounts	Student A cannot read B's rows in Supabase via app

---

6. Usability testing

6.1 Instrument

- **Survey:** `docs/SoftwareUsabilitySurvey-StudentRateUs.md` (Google Form).
- **SUS / Likert:** as embedded in form sections.

6.2 Procedure

1. Recruit ≥5 representative students (course policy).
2. Ask participants to complete **Submit Work**, **Tasks**, and **Submission Status** tasks while thinking aloud.
3. Post-session: submit Google Form.

6.3 Analysis (fill in after sessions)

Finding ID	Severity	Description	Recommendation	Owner
U-01	*(tbd)*			

---

7. Traceability matrix

Test ID	SRS reference
TC-AUTH-01..03	FR-01..04 (v2)
TC-SUB-01..03	FR-09..11 (v2)
TC-AI-01..02	FR-12..18 (v2)
TC-CT-01..06	AM-01..05, AM-10 (v3.1)

TC-ST-01..03	AM-06..09, AM-11 (v3.1)
TC-EM-01	FR-05..07 (v2)

---

8. Test summary template

Build / commit: `git rev-parse HEAD`
Environment: production / preview / local
Tester:
Date:

Area	Total	Pass	Fail	Blocked
Auth				
Submissions				
AI / Grading				
Course tasks				
Student UI				
Email				

Open defects: *(link to issue list or appendix)*

---

9. References

- `docs/SRS-Smart-Document-Evaluator-v3.md`
- `docs/SRS-Smart-Document-Evaluator-v2.md`
- `docs/SDD-Smart-Document-Evaluator-v3.md`
- `docs/SPMP-Smart-Document-Evaluator-v3.md`
- `README.md`

End of STD v3.0

Invitation Email Setup (Resend + Vercel)

This explains how to enable real Gmail-inbox invitation emails for the 43 invited students listed in `src/data/invitedStudentEmails.ts`.

The plumbing is already in place:

- `api/send-invitation-email.ts` — Vercel serverless function that calls Resend and sends the branded HTML invitation.
- `src/lib/sendInvitationEmail.ts` — fire-and-forget client helper.
- `src/components/student/StudentInvitationCard.tsx` — fires the helper exactly once per `(userId × browser)` when an invited Gmail signs in.
- `scripts/send-invitation-email-test.mjs` — CLI to send a test email without waiting for a sign-in.

All you need is a Resend API key, copied into Vercel env vars. Three steps.

---

1. Create a Resend account (free)

1. Go to https://resend.com and sign up with any email.
2. Open **API Keys** in the dashboard, click **Create API Key**, name it `smart-docs-prod`, give it the **Sending access** scope, copy the value (starts with `re_`). You will not see it again — paste it somewhere safe for a moment.
3. (Optional, recommended later) Verify a custom sending domain so the "From" address can be e.g. `invites@smartformevaluator.com` instead of the shared `onboarding@resend.dev` address. Skip for now — the shared address works out of the box and the free tier covers 100 emails / day.

---

2. Add the key to Vercel

In the Vercel dashboard, open the Smart Docs Validator project → **Settings** → **Environment Variables** and add:

Name	Value	Env
`RESEND_API_KEY`	`re_xxx_...` (the value you copied)	All
`SMARTDOCS_FROM_EMAIL`	*(optional)* e.g. `Smart Docs <invites@smartformevaluator.com>` once verified	All
`SMARTDOCS_APP_URL`	*(optional)* defaults to `https://smartformevaluator.com`	All
`SMARTDOCS_SURVEY_URL`	*(optional)* override the Rate Us Google Form URL	All

After saving, click **Redeploy** → **Use existing Build Cache** so the running deployment picks up the new env var. Future pushes inherit it automatically.

---

3. Verify it works

Option A — log in as an invited student

1. Sign in to https://smartformevaluator.com with one of the 43 Gmails in `src/data/invitedStudentEmails.ts` (e.g. `trafalgardreii@gmail.com`).
2. The yellow in-app invitation card slides in at the top-right.
3. Within a few seconds an email titled **"You've been added to Smart Docs Validator"** lands in that Gmail's inbox. (Check the **Updates** tab if it isn't in **Primary**.)

Option B — test from your terminal

From the repo root, with your Resend key set:

```
$env:RESEND_API_KEY = "re_xxx_xxx"
npm run invite:test -- trafalgardreii@gmail.com "Trafalgar Drei"
```

The script POSTs to Resend directly with the same template the deployed function uses, prints the Resend response id on success, and exits non-zero on failure (with the full Resend error message in stderr).

Troubleshooting

- `/api/send-invitation-email` **returns 500 with "RESEND_API_KEY missing"** — the env var is not on the deployment. Re-check Vercel → Settings → Env Vars and redeploy.
- **403 "This email is not on the invited list"** — the email is not in the server-side allow-list inside `api/send-invitation-email.ts`. Add it there *and* in `src/data/invitedStudentEmails.ts`, then push.
- **Email never arrives** — Resend's free tier delivers via shared IPs. Check the Gmail **Spam** folder, the **Updates** tab, and the Resend dashboard → **Logs** for the actual delivery status. `from: onboarding@resend.dev` is the default and is allowed without DNS setup.
- **Card shows up but no email** — open browser DevTools → Network tab, re-load the dashboard, and look for `POST /api/send-invitation-email`. The JSON response will contain the exact failure reason.

Why a serverless function?

The browser cannot call Resend directly because that would expose the API key to anyone who opens DevTools. The Vercel function keeps the key server-side, validates the email against the invited allow-list, and only then forwards the request to Resend. The client only ever sees a sanitised `{ ok, id?, error? }` response.

Software Usability Feedback Survey — Smart Docs Validator (Smart Document Evaluator)

Paste each section below into a Google Form (one Form section per H2 heading). All radio / Likert questions are **required** unless marked *(optional)*. Times in parentheses are rough completion estimates.

> ***Fastest path — let Apps Script build it for you (≈ 30 seconds)***
>
> *Open `scripts/google-forms/create-rate-us-form.gs`, copy the file into https://script.google.com (New project → paste → Save), pick the function `createSmartDocsValidatorSurvey`, click **Run**, authorize when prompted. The finished Form (with all sections, Likert grids, regex validation, and consent branching) lands in your Drive. The script prints the **Public URL** and **Short URL** in the Execution log — drop the short URL into `.env`:*
>
> ```
> VITE_STUDENT_RATE_US_URL=https://forms.gle/XXXXXXXX
> ```
>
> ***Manual path (5 minutes, no Apps Script)***
>
> 1. *Go to https://forms.google.com → blank form.*
> 2. *Title it `Smart Docs Validator — Software Usability Feedback Survey`.*
> 3. *Description: paste the **Form description** below.*
> 4. *For each `## Section`, click the `Add section` divider and copy each `### Question` as its own item. Use the question type listed in `Type:`.*
> 5. *Settings → Responses → **Collect email addresses** (verified), **Limit to 1 response** (optional, if you have Workspace), and **Show summary charts** to teachers only.*
> 6. *Publish → copy the public link → set `VITE_STUDENT_RATE_US_URL=<link>` in `.env` and rebuild. The floating "Rate us" pill in the student portal will now open it in a new tab.*

Form description

Help us improve **Smart Docs Validator**, the AI-assisted document evaluator your class uses for SRS / SDD / SPMP / STD and other coursework. Your feedback shapes the AI rubric, per-page Before → After feedback, and the teacher grading flow. Estimated time: **5–7 minutes**. Responses are reviewed only by the project team.

Section 1 — Consent to Participate (≈ 30 s)

Question 1 · Consent

- **Type:** Multiple choice (single answer) · **Required**
- **Question:** I have read and understand that this survey collects feedback on the Smart Docs Validator student portal, AI Evaluator, and teacher grading workflow. I voluntarily agree to participate and rate the system based on my real experience.
- **Options:**
  - I agree to participate.
  - I do not agree (closes the form).
- **Branching:** If `I do not agree` → end the form with a thank-you screen.

Section 2 — Personal Details (≈ 30 s)

Question 2 · Full name

- **Type:** Short answer · **Required**
- **Validation:** Text length ≥ 2.

Question 3 · School email

- **Type:** Short answer · **Required**
- **Validation:** *Response type → Regular expression → contains* `^[A-Za-z0-9._%+-]+@(cit\.edu|citu\.edu\.ph|.+\..+)$` · Error: "Use your campus email."

Question 4 · Role *(optional)*

- **Type:** Multiple choice (single answer)
- **Options:** Undergraduate · Graduate · Faculty · Other

Question 5 · Course / Section *(optional)*

- **Type:** Short answer · Placeholder: `e.g. IT332 — Section A`

Question 6 · How often do you use Smart Docs Validator?

- **Type:** Multiple choice (single answer) · **Required**
- **Options:** Daily · A few times a week · Weekly · A few times a month · First time today

---

Section 3 — Submission Workflow Usability (≈ 1 min)

> *Add this as a **Likert / multiple-choice grid**. Columns are the same for every row.*

Question 7 · Submission workflow — Likert grid

- **Type:** Multiple choice grid · **Required (one per row)**
- **Columns (left → right):** Strongly Disagree · Disagree · Neutral · Agree · Strongly Agree
- **Rows:**
  1. Logging in with Google OAuth was fast and reliable.
  2. Finding the right assignment in **Assignments** was easy.
  3. Uploading my document (.pdf / .docx / images) was straightforward.
  4. The status of my submission (Under review / Graded / Needs resubmission) was clear.
  5. I trusted that my uploaded file was stored securely.

Question 8 · Anything confusing about the submission flow? *(optional)*

- **Type:** Paragraph

Section 4 — AI Evaluator Quality (≈ 2 min)

> *The headline feature — keep this section sharp.*

Question 9 · AI Evaluator — Likert grid

- **Type:** Multiple choice grid · **Required (one per row)**
- **Columns:** Strongly Disagree · Disagree · Neutral · Agree · Strongly Agree
- **Rows:**
  1. The **AI rubric scores** matched my understanding of the document quality.
  2. The **Executive summary** (Strengths / Needs improvement) was specific and useful.
  3. The **Document overview & scoring** (page-range scores out of 10) felt accurate.
  4. The **Visual & diagram evaluation** (DFD, sequence, ERD, etc.) was helpful.
  5. The **Per-page Before → After** rewrite improved my draft in a meaningful way.

6. The AI cited concrete evidence (quotes / page numbers / image observations).
7. Running the evaluator was fast enough for normal use.
8. I would trust the AI suggestions before submitting a final version.

Question 10 · Which AI sections were MOST useful? *(optional)*

- **Type:** Checkboxes (multi-select)
- **Options:**
  - Rubric scores per criterion
  - Executive summary
  - Document overview & scoring (page ranges)
  - Visual & diagram evaluation (table)
  - Per-page Before → After
  - Verified correct excerpts
  - Suggested corrections (Before → After)
  - Other (write in)

Question 11 · One thing the AI got wrong on your document *(optional)*

- **Type:** Paragraph
- **Description:** Quote the AI claim or paste a screenshot link if you have it.

Section 5 — Teacher Feedback & Score Visibility (≈ 1 min)

Question 12 · Teacher review experience — Likert grid

- **Type:** Multiple choice grid · **Required (one per row)**
- **Columns:** Strongly Disagree · Disagree · Neutral · Agree · Strongly Agree
- **Rows:**
  1. I could clearly see when my work was **graded by AI**, **graded by the teacher**, or **both**.
  2. The teacher's written feedback was easy to find.
  3. The split between **AI score** and **Teacher score** made sense to me.
  4. Resubmission instructions (when present) were clear and actionable.

Question 13 · Teacher feedback comments *(optional)*

- **Type:** Paragraph

Section 6 — Performance, Accessibility & Errors (≈ 1 min)

Question 14 · Performance & access — Likert grid

- **Type:** Multiple choice grid · **Required (one per row)**
- **Columns:** Strongly Disagree · Disagree · Neutral · Agree · Strongly Agree
- **Rows:**
  1. Pages load quickly on my device.
  2. The interface looks good on **mobile / small screens**.
  3. Text size and contrast were comfortable to read.
  4. I rarely (or never) hit errors or unexpected behavior.

Question 15 · Which device do you mostly use? *(optional)*

- **Type:** Multiple choice (single answer)
- **Options:** Laptop · Desktop · Tablet · Phone · Mix

Question 16 · Browser *(optional)*

- **Type:** Multiple choice (single answer)
- **Options:** Chrome · Edge · Firefox · Safari · Brave · Other

Question 17 · Describe a bug or error you saw *(optional)*

- **Type:** Paragraph

Section 7 — Overall Recommendation (≈ 30 s)

Question 18 · NPS — Likelihood to recommend

- **Type:** Multiple choice (single answer) · **Required**
- **Question:** How likely are you to recommend Smart Docs Validator to a classmate?
- **Options:**
  - Very Unlikely (0–2)
  - Unlikely (3–4)
  - Neutral (5–6)
  - Likely (7–8)
  - Very Likely (9–10)

Question 19 · Single biggest improvement *(optional)*

- **Type:** Paragraph
- **Description:** If we could fix only ONE thing in the next release, what should it be?

Question 20 · Anything else you want to tell us? *(optional)*

- **Type:** Paragraph

After-submit screen

Confirmation message:

> ***Thanks — we got your feedback.*** *Your responses go directly to the Smart Docs Validator team. If you reported a bug, a teacher or admin may follow up using your campus email.*

Wiring it into the student portal

1. Publish the form (top-right **Send** → **Link** icon → copy short URL).
2. Edit your project `.env`:

   ```
   VITE_STUDENT_RATE_US_URL=https://forms.gle/XXXXXXXX
   ```

3. Restart `npm run dev` (or rebuild for production).
4. The floating maroon **"Rate us"** pill in the bottom-right corner of every student page will now open this form in a new tab. Teachers and admins never see the pill — it is gated by `user.role !== 'teacher' && 'admin'` in `src/components/Layout.tsx`.
5. If `VITE_STUDENT_RATE_US_URL` is left empty, the pill falls back to the built-in in-app rating modal (`src/components/student/StudentRateUsButton.tsx`).